DNS FROM THE FIRST PRINCIPLES

ALEXY KHRABROV

FIRST PAIR PRESS

DNS from First Principles

Names, Packets, Authority, Recursion, and the Design of rgbdns

Alexy Khrabrov

2026-08-22

Contents

Preface

DNS is often introduced as "the Internet's phone book." That metaphor is useful for about a minute. A phone book is one database, published in editions, mapping people to telephone numbers. DNS is a distributed protocol for finding typed, time-limited statements in a delegated tree. It has many writers, many readers, caches between them, multiple transports, and rules for proving both presence and absence. Names can point to addresses, but they can also identify mail exchangers, service endpoints, authoritative servers, cryptographic keys, and arbitrary text.

This book develops DNS from those underlying problems. The first half builds a mental model independent of any implementation. The second half walks through rgbdns, a memory-safe Rust reimplementation of the djbdns suite. The aim is not only to explain what each program does, but why its boundaries look the way they do: immutable compiled data for authority, a separate recursive cache, small diagnostic clients, foreground daemons, and stream-oriented logging.

The code is the final authority for rgbdns behavior. This book describes rgbdns 0.6.3 and book version 0.4.0 as built on 2026-08-21.

1 The problem DNS solves

1.1 Identity is not location

A network delivers packets to addresses. Humans and applications want stable identities. Those two things should not be fused.

Suppose a service is reached at `192.0.2.8`. If that address is embedded in every configuration, moving the service requires changing every client. A name such as `api.example` introduces indirection:

```
application → api.example → 192.0.2.8 → packets
```

Indirection has a cost: another system must answer the middle question. Its benefit is that the service owner can change the answer without changing the application. DNS is the globally deployed mechanism for this indirection.

The mapping is not a function from one name to one address. One name may have several addresses. The answers may differ by client location. A mail domain may name several mail exchangers with preferences. A service may delegate a subtree to another organization. The useful abstraction is therefore:

```
(owner name, record type, class) → a set of resource records
```

The owner and type together select an RRset. “RRset” means all resource records with the same owner, type, and class. Implementations should normally treat the set as a unit because caches and DNSSEC signatures do.

1.2 Requirements that pull in different directions

A global naming system must satisfy conflicting demands:

- It must scale without one central database receiving every query.
- Different organizations must control different parts of the namespace.
- Changes must propagate, but cached answers are essential for performance.
- Replies should usually fit in one datagram, but some answers are large.
- Old implementations must coexist with protocol extensions.
- A client needs to distinguish “no such name” from “that name has no record of this type.”
- Operators need a way to transfer complete zones and to diagnose individual exchanges.

DNS answers these demands with hierarchy, delegation, caching lifetimes, compact binary messages, UDP plus TCP, explicit result codes, and typed records. Many operational surprises are direct consequences of those design choices rather than random quirks.

1.3 Roles, not just “DNS servers”

The phrase “DNS server” hides several jobs.

An **authoritative server** publishes data for zones it controls. It answers from configured facts and does not chase referrals on behalf of a client.

A **recursive resolver** accepts a question from a stub client, follows the delegation chain, validates and caches what it learns, and returns a final answer.

A **stub resolver** is the client-side library or program that sends a recursive query to a configured resolver.

A **forwarder** sends selected questions to another resolver rather than performing iteration itself.

Keeping these roles distinct is both conceptual hygiene and a security boundary. An authoritative daemon does not need a large mutable Internet-fed cache. A recursive resolver does not need the private machinery used to edit a zone. rgbdns follows the djbdns design and runs authority and recursion as different programs.

2 Names form a delegated tree

2.1 Labels and the root

A DNS name is a sequence of labels. In the presentation form `www.example.com.`, the dots separate the labels `www`, `example`, and `com`. The final dot represents the root's empty label. Reading from right to left walks from general to specific:

```
.                      root
└── com.                top-level domain
    └── example.com.    delegated domain
        └── www.example.com.
```

The absolute name has a wire limit of 255 octets, including length bytes and the terminating root label. Each ordinary label is at most 63 octets. DNS names are not inherently UTF-8 strings. Internationalized names are normally converted by applications into ASCII-compatible labels before DNS sees them.

DNS comparison is case-insensitive for ordinary ASCII letters, although the original spelling can be preserved. A robust implementation therefore needs a canonical comparison rule without losing the bytes required for encoding.

In rgbdns, `src/name.rs` represents a name as a vector of byte-vector labels. Construction validates label and total lengths. Parsing accepts the familiar dotted form and backslash escapes. The type provides parent and subdomain operations, case-insensitive ordering, display formatting, and wire encoding. Making invalid names difficult to construct removes repeated checks from the rest of the system.

2.2 Zones are administrative cuts

The namespace is one tree; a zone is an administratively served portion of that tree. The two are not identical.

The zone `example.com.` might contain records for `www.example.com.` and `mail.example.com.`, then delegate `research.example.com.` to other servers. The child remains below `example.com.` in the namespace but is outside the parent zone's authoritative contents.

A delegation is expressed by NS records at the cut. If a named server lies inside the delegated child, a resolver cannot first resolve that server's name through the child—it needs its address in order to reach the child. The parent therefore supplies an address record called **glue**. Glue is navigation data, not an assertion that the parent is authoritative for every fact about the host.

The root zone delegates top-level domains. A cold recursive resolver starts with a small configured set of root server addresses, asks the root where to find a top-level domain, asks that domain where to find the next child, and continues.

2.3 Wildcards are synthesis rules

A wildcard such as `*.example.com.` does not mean "return this record for every name ending in example.com." It participates only when the queried name does not exist, and the closest-encloser rules determine which wildcard, if any, can synthesize an answer. Existing intermediate names can block a wildcard.

rgbdns stores wildcard records under their literal wildcard owner and its zone lookup searches from the queried name toward the closest existing ancestor. It synthesizes the queried owner in returned records. This is more precise than a string suffix match and is one reason the `Zone` abstraction tracks known nodes in addition to records.

3 Resource records: typed facts with lifetimes

3.1 The common envelope

Every resource record has:

- an owner name;
- a numeric type;
- a class, almost always Internet class `IN`;
- a time to live, or TTL;
- type-specific data called RDATA.

The TTL is a lease offered to caches. If an authoritative server returns a TTL of 300 seconds, a cache may reuse that answer for at most five minutes before refreshing it. The TTL does not schedule a change and does not guarantee that every cache holds the answer for the full interval. It establishes an upper bound.

Changing a record and then lowering its TTL is too late for clients that already cached the older, longer lease. Planned migrations lower the TTL at least one old-TTL interval before the change, wait, make the change, and later raise it.

3.2 Core types

A maps an owner to an IPv4 address. **AAAA** maps it to an IPv6 address. Several records at one owner form an address RRset; DNS does not promise that clients use them in listed order.

NS names an authoritative server for a zone or delegation.

SOA, the start of authority, identifies the zone and carries operational parameters: primary server, responsible mailbox, serial, refresh, retry, expire, and negative-cache values. A secondary compares serial numbers to decide whether a transfer is needed. Serial arithmetic wraps in a defined 32-bit space, so blindly treating it as an ordinary integer can be wrong near the boundary.

CNAME says that its owner is an alias of another name. Except for DNSSEC and narrowly specified metadata, an owner with CNAME should not also hold unrelated data. A resolver follows the chain while defending against loops and excessive depth.

ANAME is not a standard DNS record type in rgbdns. It is authoritative server configuration that copies the address meaning of another name onto an owner. This distinction matters: clients receive ordinary A or AAAA records, not an ANAME record. ANAME can therefore provide address aliasing at a zone apex without violating CNAME exclusivity.

MX names a mail exchanger and gives it a preference. Lower numbers are preferred. The target is a name, not an address.

PTR provides a name-valued reverse mapping. IPv4 reverse names live below `in-addr.arpa.` with octets reversed. IPv6 reverse names live below `ip6.arpa.` with hexadecimal nibbles reversed.

TXT carries one or more length-delimited byte strings. Presentation formats often make it look like one free-form string, but the wire format retains segments.

SRV names a service endpoint with priority, weight, port, and target.

CAA constrains which certification authorities may issue certificates for a domain.

OPT is not ordinary zone data. It is a pseudo-record used by EDNS to negotiate UDP payload size and carry extension flags and options.

rgbdns models these forms with `RecordType`, `Record`, and the `RData` enum in `src/packet.rs`. Known structured types receive structured variants. Unknown types can remain opaque where the format permits, preserving extensibility without confusing untrusted lengths with trusted objects.

3.3 Additional data is an optimization

If an answer contains MX, NS, or SRV targets, the server may include associated A and AAAA records in the additional section. This can save queries. It does not change which RRset directly answers the question, and a resolver must apply the correct credibility rules rather than trusting unrelated additional data.

The rgbdns authoritative response path collects target names from those record types and adds locally available addresses. It de-duplicates targets before lookup and preserves the distinction between answers and helpful additionals.

4 Messages on the wire

4.1 The twelve-byte header

A DNS message begins with a fixed twelve-byte header:

```
0                   1                   2                   3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-------------------------------+-------------------------------+
|              ID               |             flags             |
+-------------------------------+-------------------------------+
|          question count       |          answer count         |
+-------------------------------+-------------------------------+
|         authority count       |         additional count      |
+-------------------------------+-------------------------------+
```

The transaction ID lets a client associate a response with a query. Important flags include QR (query versus response), opcode, AA (authoritative answer), TC (truncated), RD (recursion desired), RA (recursion available), and the four-bit response code.

The four following sections contain questions, answers, authority records, and additional records. A normal question carries a name, requested type, and class. Resource-record sections add TTL, RDATA length, and RDATA.

All multibyte integers are network byte order. Every count and length comes from an untrusted peer. A decoder must prove that bytes exist before reading them, cap allocations, reject invalid labels and pointers, and finish with a coherent message rather than a partially trusted structure.

4.2 Name compression

Repeating full names would waste scarce datagram space. DNS permits a name suffix to be replaced by a two-byte pointer whose high bits are `11` and whose remaining bits are an offset earlier in the message.

Compression turns name decoding into graph traversal. A malicious packet can contain a pointer loop, excessive indirection, or an offset outside the packet. A safe decoder tracks visited offsets or imposes a strict jump bound, checks every target, and separately enforces the expanded 255-octet name limit.

rgbdns's `Reader` in `src/packet.rs` keeps all reads within a borrowed byte slice. Name decoding validates pointer targets and bounds traversal. Record decoding confines each RDATA parser to the declared RDLENGTH. EDNS option iteration likewise checks the option header and value before advancing.

The `Writer` performs the reverse operation. Encoding is fallible: counts must fit 16 bits, names and RDATA must fit their fields, and the result must remain valid. This symmetry—decode into typed data, manipulate typed data, encode with checks—is the packet layer's central safety property.

4.3 Errors are protocol results

Several results that sound similar are materially different:

- **NOERROR with answers**: the requested RRset exists.
- **NOERROR without answers**, often called NODATA: the name exists but the requested type does not.
- **NXDOMAIN**: the queried name does not exist.
- **SERVFAIL**: the server could not safely complete processing.
- **REFUSED**: policy forbids the operation.
- **FORMERR**: the message is malformed.
- **NOTIMP**: the requested opcode is unsupported.

Negative answers normally include the zone's SOA so resolvers can cache the negative result. Confusing NODATA with NXDOMAIN can suppress other valid types at the same name.

rgbdns expresses authoritative lookup outcomes as `Lookup::Answer`, `Referral`, `NoData`, `NxDomain`, and `Refused`. That internal sum type forces the response builder to handle each protocol meaning explicitly.

5 UDP, TCP, EDNS, and truncation

5.1 Why there are two transports

Classic DNS uses UDP for ordinary queries because one request and one response need no connection setup. Traditional UDP DNS assumed a 512-byte message. Larger answers set TC, telling the client to retry over TCP. TCP frames every DNS message with a two-byte length.

Zone transfers use TCP. Modern responses—especially DNSSEC responses—often need more than 512 bytes, so EDNS lets a client advertise a larger UDP receive size through an OPT pseudo-record. Internet paths can still drop fragmented UDP packets. A commonly conservative payload is 1232 bytes, large enough for useful DNSSEC answers while fitting the IPv6 minimum MTU without fragmentation under normal headers.

TCP is not merely an emergency protocol. Firewalls that assume DNS is UDP-only break standards-compliant resolution.

5.2 Truncation must preserve a valid message

Cutting the last bytes off an encoded message creates a malformed packet. Correct truncation removes complete records, sets TC, updates section counts, and re-encodes. A useful removal order discards nonessential additional data before authority and answer data. OPT sometimes needs special treatment because it carries the EDNS response.

`src/server.rs` calculates a response limit from the caller's transport limit and the client's EDNS advertisement. It caps advertised UDP size, rejects multiple OPT records, responds to unsupported EDNS versions, and constructs a full typed response. If encoding exceeds the limit, `truncate` sets TC and removes complete records in a defined order until the packet fits. The same core response logic serves UDP and TCP without treating TCP as a giant UDP datagram.

6 Authority: answering from owned facts

6.1 Finding the closest relevant authority

For a question `(name, type)`, an authoritative server determines:

1. whether the name lies in a served zone;
2. whether a delegation cut is closer than that zone's apex;
3. whether the exact name exists;
4. whether the requested RRset exists;
5. whether a CNAME or wildcard changes the answer;
6. which SOA proves a negative result.

A query beneath a delegated child should produce a referral, not an authoritative negative answer from the parent. A query outside all configured zones should normally be refused. These boundary checks matter more than a simple map lookup.

rgbdns's `Zone` stores records in a `BTreeMap<Name, Vec<Record>>`, authoritative apices and delegation owners in ordered sets, and separate metadata for location and activation. Lookup walks name ancestry, recognizes cuts, filters visible records, applies exact-name and wildcard rules, and returns the typed `Lookup` outcome.

The response builder then:

- copies the query ID and relevant RD bit;
- marks authoritative answers with AA;
- expands CNAME chains with a 16-hop limit and visited-name set;
- adds address records for NS, MX, and SRV targets;
- clears AA on referrals;
- adds the SOA to negative answers;
- maps malformed, unsupported, and policy cases to protocol response codes.

The finite CNAME bound and visited set are deliberate denial-of-service and correctness controls. A cyclic zone must not turn one datagram into unbounded work.

6.2 tinydns data as a source language

djbdns uses a compact line-oriented zone source called `data`. The first character selects a record form. Common forms include:

Prefix	Meaning
`.`	zone authority plus NS and address data
`&`	delegation NS and optional glue
`Z`	explicit SOA
`=`	A plus matching reverse PTR
`+`	A only
`6`	AAAA plus reverse PTR forms
`3`	AAAA only
`@`	MX and optional exchanger address
`C`	CNAME
`A`	private ANAME (flattened A/AAAA alias)
`^`	PTR
`'`	TXT
`S`	SRV
`:`	generic record
`%`	client-location mapping

Fields are colon-separated with octal escapes for bytes that would otherwise be ambiguous. Optional fields carry TTL, timestamp, and location information. The format is terse because it was designed for mechanical generation as well as hand editing.

`Zone::parse` reads this language line by line. It ignores blank, comment, and disabled lines; reports the failing line number; expands convenience forms into ordinary typed records; validates IPv4, flat 32-digit IPv6, names, numeric ranges, and escaped bytes; and records authoritative and delegation structure. When an SOA serial is omitted, file loading derives a nonzero default from the source modification time.

Timestamp fields use TAI64-style cutoffs. Depending on the marker, a record can be visible before or after a specified instant. Location codes select records using configured client IPv4 prefixes. rgbdns carries that metadata beside the record and evaluates it at lookup time.

6.3 ANAME and apex address flattening

A zone apex necessarily owns SOA and NS data. A CNAME owner, by contrast, cannot also own ordinary records. A literal apex CNAME would therefore make the zone internally contradictory: the alias rule says the owner has no other data while the authority rules require other data at that same owner.

Hosted sites still need a way for `example.com` to track addresses controlled by a platform such as `customer.blog-host.example`. DNS providers commonly call the solution CNAME flattening, ALIAS, or ANAME. The authoritative server resolves the configured target itself and publishes the resulting addresses under the configured owner.

rgbdns calls this feature **ANAME** and uses the private `A` source marker:

```
# Authority and nameserver address.
.example.com:192.0.2.53:ns1.example.com

# ANAME owner:target:maximum-ttl
Aexample.com:customer.blog-host.example:300

# Other apex data remains independent.
@example.com::mail.example.com:10:3600
'example.com:v=spf1 -all:3600
```

The form is:

```
Aowner:target:maximum-ttl
```

The TTL field is optional and defaults to 300 seconds. It is a ceiling, not a promise to extend the target's lifetime. If the upstream address has 45 seconds remaining and the ANAME limit is 300, rgbdns returns 45. If the upstream has 900 seconds remaining, rgbdns returns no more than 300.

ANAME is stored separately from ordinary `Record` values. It can coexist with SOA, NS, MX, TXT, CAA, and other non-address data. Zone validation rejects:

- A, AAAA, or CNAME data at the same owner;
- a wildcard ANAME owner;
- an owner that targets itself;
- different ANAME targets at one owner;
- a zero TTL.

ANAME is private configuration rather than a standard DNS RR type, so a plain AXFR cannot represent it. rgbdns peers negotiate preservation explicitly: `axfr-get` places private EDNS option 65001 with the `RGA1` version token in the AXFR request. An rgbdns primary then inserts private-use TYPE65401 records whose TTL is the ANAME cap and whose payload is the token followed by the target's uncompressed wire name. The receiving rgbdns validates that payload and reconstructs the `Aowner:target:ttl` directive before compilation.

An ordinary AXFR request does not receive TYPE65401. This keeps standard secondaries free of private metadata, but it also means they do not reproduce ANAME behavior. Delegate an ANAME-backed zone only to upgraded rgbdns peers, or use standard address records when non-rgbdns secondaries must serve it.

The server only applies ANAME to A and AAAA questions. SOA, NS, MX, TXT, CAA, and all other questions continue through normal authoritative lookup. ANAME also does not override a delegation cut: a name beneath a delegated child still produces a referral from the parent.

For an address question, the response path is:

1. establish that ordinary authoritative lookup reaches the ANAME owner and does not cross a delegation;
2. query the configured recursive resolver for the target and requested address family;
3. validate response identity and framing;
4. follow only a connected CNAME chain beginning at the configured target;
5. collect the terminal A or AAAA RRset;
6. replace each terminal owner with the ANAME owner;
7. cap the remaining TTL and return an authoritative answer.

For example, an upstream result such as:

```
customer.blog-host.example. 180 IN CNAME edge.host.example.
edge.host.example.          120 IN A     192.0.2.80
```

becomes:

```
example.com.                120 IN A     192.0.2.80
```

The CNAME is deliberately absent. Consumers see a conventional authoritative address RRset at the apex.

The resolver cache is shared by requests handled by one server process. Positive entries expire with the upstream chain's shortest relevant TTL. Negative results use the authority SOA's negative TTL when available and 60 seconds otherwise. The configured ANAME ceiling is applied when constructing each response, so two owners may safely share a target while using different TTL policies. Concurrent misses for one target and address family are coalesced into one upstream query. A failed lookup is retained for five seconds, during which matching requests fail without starting another recursive chain. This short circuit limits retry storms and cross-provider ANAME loop amplification without turning a transient failure into long-lived negative DNS data.

Resolution is bounded in the same spirit as the rest of rgbdns:

- CNAME chains stop after 16 links;
- visited names detect cycles;
- no more than 64 terminal addresses are accepted;
- conflicting CNAME targets are rejected;
- upstream SERVFAIL and other resolver errors become authoritative SERVFAIL, not false NODATA;
- concurrent misses share one lookup and failures suppress immediate retries;
- A and AAAA are cached independently.

`DNSCACHEIP` selects one or more recursive endpoints, separated by commas. Each endpoint may be an IP address using port 53 or an explicit socket address. Without it, rgbdns reads `/etc/resolv.conf`. A local validating `dnscache` is the preferred upstream when operators want DNSSEC validation and a cache shared with other local DNS work:

```
DNSCACHEIP=127.0.0.1:5354 IP=0.0.0.0 PORT=53 tinydns
```

ANAME metadata survives `tinydns-data` compilation through private CDB entries; it is not encoded as a made-up public RR type. This retains the source semantics across text and CDB operation without teaching ordinary DNS clients about a private wire format.

Standard AXFR has no interoperable way to describe this private policy. rgbdns therefore emits no ANAME metadata to an ordinary AXFR client. Upgraded rgbdns peers can explicitly negotiate the experimental `RGA1` extension: the requester sends EDNS option 65001 and the primary returns validated private-use TYPE65401 records only for that transfer. The receiver restores the source directive and resolves the target independently. A conventional secondary still receives only standard DNS records and cannot reproduce ANAME behavior. Operators must delegate an ANAME-backed zone only to upgraded rgbdns peers unless they materialize standard A and AAAA records instead.

6.4 CDB: compile once, read predictably

The traditional `tinydns-data` compiles text into a constant database, CDB. The serving process reads the compiled file instead of reparsing editable text for every startup or query. Compilation also enables atomic replacement: write and validate a new file, then rename it into place.

rgbdns's `src/cdb.rs` preserves the djbdns key/value layout for ordinary records and uses a private, NUL-prefixed key namespace for ANAME metadata. `compile` serializes typed records and metadata; `load` reads entries and reconstructs a `Zone`. The loader does not trust the database merely because it is local. It bounds file and entry sizes, validates keys, checks record layouts, decodes names and RDATA through explicit lengths, and rejects malformed data.

This is an important general rule: compiled configuration is still input. It may be truncated by a failed copy, generated by an older tool, or replaced by an attacker with filesystem access. Memory safety should not depend on the provenance story being perfect.

7 Recursion: discovering an answer

7.1 Iteration from the root

A recursive resolver turns one client request into a bounded sequence of queries. For `www.example.com. A`, a cold lookup is approximately:

```
stub → recursive resolver
          ├─ root:       who serves com?
          ├─ com server: who serves example.com?
          └─ example:    what is www.example.com A?
     ← final answer
```

The resolver follows referrals, resolves nameserver addresses when glue is insufficient, handles aliases, retries servers and transports, and detects loops. It caches useful RRsets so later clients may skip most of this path.

Root hints are not answers to every name. They are bootstrap addresses for reaching the root authority. They need periodic maintenance because the set can change, though names and anycast make changes infrequent.

7.2 The cache is part of correctness

A cache key includes at least name, type, and class. A cached positive RRset expires according to TTL. Negative results also have bounded lifetimes derived from SOA data. Delegation and nameserver-address caches help the iterative algorithm navigate efficiently.

Capacity is as important as time. An attacker can generate endless distinct names. An unbounded cache converts traffic into memory exhaustion. A practical resolver bounds response cache bytes, nameserver cache entries, recursion depth, referral work, packet sizes, concurrent operations, and timeouts.

`src/bin/dnscache.rs` uses Hickory's recursive zone handler inside rgbdns's process and policy shell. It configures:

- randomized query-name letter case;
- a bounded response cache, defaulting to 16 MiB;
- a bounded nameserver cache;
- bounded ordinary and nameserver recursion depth;
- a 1232-byte EDNS payload;
- UDP and TCP listeners;
- loopback-only clients by default, expanded through `ALLOW_NETS`.

Configuration values are parsed with explicit minimums and maximums. A typo such as an enormous cache size fails startup rather than silently allocating an operator's mistake.

7.3 Forward zones and djbdns roots

Private namespaces and split DNS often need selected suffixes sent to specific servers. rgbdns reads forward-zone configuration from the environment and the djbdns-style `ROOT/servers` directory. The filename identifies a suffix and the file lists bounded server addresses.

The special `servers/@` file represents root servers. Hickory consumes a root hints file in master-file syntax, so `PreparedRoots` translates djbdns's plain address list into a private temporary file. Creation uses restrictive permissions and cleanup occurs when the prepared object is dropped. This adapter preserves the external configuration contract without weakening the library boundary.

Forwarded private zones disable strict case-randomization response matching because legacy authorities may canonicalize owner case. They retain TCP retry and a bounded cache. This is a scoped compatibility decision, not a global removal of query hardening.

8 DNSSEC: authenticating the chain

8.1 What ordinary DNS cannot prove

Transaction IDs, source ports, and query-case randomization make blind spoofing harder, but they do not cryptographically establish who published an RRset. DNSSEC adds signatures and a chain of trust.

A zone signs RRsets with private keys and publishes DNSKEY records. A parent publishes a DS digest that identifies a child key. Starting from a configured root trust anchor, a validating resolver can authenticate the root DNSKEY, then a top-level domain's DS and DNSKEY, and so on to the answer.

RRSIG authenticates an RRset over a validity interval. DS links parent to child. NSEC or NSEC3 authenticates nonexistence by proving gaps in the ordered namespace. DNSSEC provides origin authentication and integrity; it does not encrypt queries or hide names.

Validation outcomes matter:

- **secure**: a valid chain reaches the answer;
- **insecure**: the chain proves that the child is unsigned;
- **bogus**: signatures or proofs fail;
- **indeterminate**: validation cannot be completed safely.

A resolver must not turn bogus data into a normal answer merely to improve availability. Clock correctness also becomes a dependency because signatures have inception and expiration times.

8.2 Two distinct DNSSEC roles

DNSSEC has a consumer side and a producer side. A validating recursive resolver follows DS and DNSKEY records from a trust anchor and rejects bogus answers. An authoritative operator creates keys, signs each stable RRset, publishes proofs of nonexistence, and arranges for the parent to publish the matching DS.

rgbdns implements both roles, but deliberately keeps them separate. `dnscache` uses a static root trust anchor and DNSSEC validation. Validation and NSEC3 work receive bounded caches and iteration policies. A failed validation becomes a resolution failure rather than an unchecked answer.

Authoritative signing is an offline publication pipeline. `tinydns` never opens a private key and never signs in response to a packet. It serves DNSKEY, RRSIG, NSEC, and DS records already present in its CDB. This is the same compile-then-serve division used by ordinary tinydns data, extended to signed snapshots.

8.3 DNSSEC in the tinydns posture

The design goal is not a control plane. It is a set of small programs and visible files:

Program	One bounded job
`rgbsec-keygen`	create one ECDSA P-256 key and print its policy line
`rgbsec-sign`	turn tinydns text into inspectable signed tinydns text
`rgbsec-data`	sign and compile directly to a CDB
`rgbsec-ds`	derive the parent DS record from one policy line
`rgbsec-check`	verify RRsets, signatures, NSEC cycles, and remaining lifetime

There is no signing daemon, key database, online RPC service, or implicit key generation in the authority. Each transformation writes beside its destination and renames only after success. A failed materialization, signature, compile, reload, or verification leaves the last-known-good CDB active.

The feature is optional. Without a DNSSEC policy and `/etc/rgbdns/dnssec.env`, `tinydns-data`, `tinydns`, ACME, ANAME, AXFR, and secondary synchronization retain the djbdns-compatible path. Enabling signing for one zone does not force every zone in the same CDB to become signed.

8.4 One line per zone

The policy file is intentionally close to tinydns data: comments, blank lines, and exactly one disposition line for every authoritative zone. `K` means sign; `U` means explicitly unsigned:

```
Kexample.com.:/etc/rgbdns/keys/example.com.pk8:13:1209600:86400:3600
Ulegacy.example.
```

The signed line has this shape:

```
Kzone:keyfile:algorithm:validity:refresh:inception-skew
```

Algorithm 13 is ECDSA P-256 with SHA-256. The example signs for fourteen days, requires at least one day of remaining lifetime, and moves inception one hour into the past to tolerate modest clock skew. The key path is absolute. Missing, duplicate, or conflicting dispositions stop publication; omission never means "probably unsigned." A `U` zone is also checked for stale DNSKEY, RRSIG, NSEC, or NSEC3 records.

This fail-closed coverage rule is what makes mixed snapshots understandable. The policy is not merely a list of zones that happened to be signed today. It is the security disposition of the complete authoritative source.

8.5 Minimal signed primary

Begin with ordinary tinydns source. The signer preserves the SOA serial, so the source producer must increment it when the signed publication first becomes visible:

```
Zexample.com:a.ns.example.com:hostmaster.example.com:2026082101:16384:2048:1048576:2560:3600
&example.com:192.0.2.53:a.ns.example.com:3600
+example.com:192.0.2.80:3600
+www.example.com:192.0.2.80:3600
```

Create the key only on the primary. The command refuses to overwrite an existing file and creates the PKCS#8 key with mode 0600:

```
sudo install -d -o root -g root -m 0700 /etc/rgbdns/keys
sudo rgbsec-keygen example.com \
  /etc/rgbdns/keys/example.com.pk8 \
  | sudo tee /etc/rgbdns/dnssec
sudo chown root:rgbdns /etc/rgbdns/dnssec
sudo chmod 0640 /etc/rgbdns/dnssec
```

Keep an encrypted, recoverable copy of the key outside the server. The public policy may be readable by the checker, but the key directory remains root-only. The authority process and the secondary never need access.

Configure the packaged primary with the complete existing role options plus the policy:

```
sudo rgbdns-setup primary \
  --data /srv/dns/rgbdns.data \
  --data-drop /srv/dns/rgbdns.data \
  --data-drop-owner dns-publisher \
  --listen-ip 0.0.0.0 \
  --allow-nets 10.0.2.10/32 \
  --dnssec-policy /etc/rgbdns/dnssec
```

Setup performs the first privileged publication and enables two supervised jobs. `rgbdns-dnssec-publish.timer` refreshes materialized data and signatures every twelve hours. `rgbdns-dnssec-check.timer` checks the active CDB hourly. The root publisher can read the key; the checker runs as `rgbdns` and verifies only public state.

The same stages can be inspected manually in a scratch directory:

```
ln -s /etc/rgbdns/dnssec dnssec
rgbsec-sign data data.signed
rgbsec-data data data.cdb
rgbsec-check data.cdb /etc/rgbdns/dnssec
rgbsec-ds 'Kexample.com.:/etc/rgbdns/keys/example.com.pk8:13:1209600:86400:3600'
```

`rgbsec-check` emits one tab-separated line per `K` zone:

```
example.com.  2026082101  53856  20260904023527  1204490  ok
```

The exact times and key tag vary. The important operational contract is the exit status and final `ok`: every authoritative RRset verifies, the NSEC cycle is complete, and the earliest signature remains outside the refresh window.

8.6 Mixed zones and a keyless secondary

One CDB can serve a stable signed zone beside a zone that must remain unsigned:

```
Kexample.com.:/etc/rgbdns/keys/example.com.pk8:13:1209600:86400:3600
Uacme-live.example.
```

The source contains both SOAs and both NS sets. `rgbsec-data` signs only `example.com`; it preserves ordinary records, qualified records, cutoffs, and ANAME directives in the `U` zone. This behavior is explicit rather than inferred from the absence of a key.

A secondary needs neither the policy nor the private key. Configure it in the ordinary way:

```
sudo rgbdns-setup secondary \
  --zone example.com \
  --zone acme-live.example \
  --primary 10.0.1.10 \
  --listen-ip 0.0.0.0
```

AXFR carries DNSKEY, RRSIG, NSEC, and DS as standard resource records. The secondary validates the transfer framing, compiles the received snapshot, and serves the same finished signatures. A transfer failure retains the last good zone. This is a useful security boundary: compromise of a serving secondary does not disclose the signing key.

Before touching the parent, query every delegated authority over UDP and TCP:

```
for server in a.ns.example.com b.ns.example.com; do
  dig +dnssec +norecurse @"$server" example.com DNSKEY
  dig +dnssec +norecurse @"$server" example.com A
  dig +dnssec +norecurse @"$server" \
    "MiXeD-$(date +%s).example.com" A
  dig +tcp +dnssec +norecurse @"$server" \
    "tcp-$(date +%s).example.com" A
done
```

Require matching SOA serials and DNSKEYs. Positive answers need the data RRset and its RRSIG. A nonexistent mixed-case name must return NXDOMAIN with NSEC and RRSIG. The mixed case is deliberate: validating resolvers use DNS 0x20 case randomization, and authoritative suffix matching must remain ASCII case-insensitive. A query for an absent type at an existing name tests signed NODATA separately from NXDOMAIN.

8.7 ANAME and ACME boundaries

A signed answer cannot be synthesized after signing. The publication graph is therefore visible:

```
source -> ACME overlay -> selected ANAME materialization
       -> rgbsec-data -> rgbsec-check -> atomic activation
```

For an ANAME in a `K` zone, `aname-materialize` resolves the target before signing and writes ordinary A and AAAA RRsets under the configured TTL ceiling. The secondary receives those addresses and signatures; it does not resolve the target independently for that signed snapshot. ANAME directives in `U` zones retain the ordinary runtime behavior.

ACME is a privilege-boundary question, not only a record-format question. An ACME-managed `U` zone can keep using the unprivileged live overlay even while another zone in the same CDB is signed. Inline updates to a `K` zone would need an explicitly designed privileged publisher that acknowledges an update only after a new signed CDB is durable. `rgbdns-setup` does not invent that escalation. Without such a hook, a signed ACME policy fails closed.

The simplest arrangement is often a small unsigned validation child:

```
_acme-challenge.example.com.  NS  ns1.validation.example.
```

The production lesson is conservative: choose the first DNSSEC pilot from a stable zone with ordinary A or AAAA data, no live ACME overlay, and no ANAME. Move dynamic zones only after their signed publication boundary is explicit.

8.8 Activating the parent DS

Signing the child does not complete the chain. The parent must publish a DS that matches the active child DNSKEY. Derive it from the installed policy rather than transcribing public-key material by hand:

```
sudo rgbsec-ds "$(
  sudo grep '^Kexample[.]com[.]:' /etc/rgbdns/dnssec
)"
```

The output is a presentation-format record:

```
example.com. IN DS <key-tag> 13 2 <64-hex-sha256-digest>
```

That line is illustrative; use the output for the actual key. Registrar forms usually ask for four fields: key tag, algorithm 13, digest type 2, and the SHA-256 digest. Publish it only after every nameserver in the parent delegation serves the matching signed snapshot. A stale third-party secondary is enough to make validation intermittent.

Check the parent directly, then use several independent validating resolvers:

```
dig +short +nosplit example.com DS

for resolver in 1.1.1.1 9.9.9.9 8.8.8.8; do
  dig +dnssec @"$resolver" example.com A
  dig +dnssec @"$resolver" \
    "MiXeD-$(date +%s).example.com" A
done
```

A secure positive response is `NOERROR` with the AD flag. A secure negative response is NXDOMAIN with AD. Querying a fresh name avoids mistaking an old negative cache entry for current behavior. The parent DS, child DNSKEY, and signatures can be correct while a resolver still exposes a transport, delegation, case-handling, or denial-proof defect; end-to-end testing is what turns a collection of records into evidence.

8.9 Lifecycle and safe rollback

DNSSEC adds time to the authority contract. Monitor the publisher and checker, the earliest RRSIG expiration, key tag, parent DS, primary/secondary serials, and both positive and negative validation. Restart each authority during the pilot and prove that the last signed CDB remains available. Before the first signature window closes, observe an automatic refresh and verify the new inception and expiration on both authorities.

Version 0.6.3 deliberately supports one active combined signing key per zone. It does not pretend that replacing a key file is a rollover protocol. A future multi-key release must represent publication, DS overlap, retirement, and recovery as explicit states.

Rollback order is asymmetric. Before DS activation, the operator may return to the unsigned path because no validator expects signatures. After DS activation:

1. remove the DS at the parent;
2. wait through the parent DS TTL, negative caches, and relevant resolver caches;
3. confirm that independent resolvers no longer observe the DS; and only then
4. disable signing and publish unsigned data.

Removing signatures while the DS remains visible converts the zone from secure to bogus. Keep the retired private key backed up until the DS removal and cache window are conclusively complete.

8.10 Constraints are part of the design

rgbdns uses NSEC rather than NSEC3 for authoritative signing. NSEC is smaller, simpler, and avoids iteration and opt-out machinery; it also makes zone names enumerable. Location-dependent and time-qualified data cannot be signed because one owner and type must identify one stable RRset. Pre-existing DNSKEY, RRSIG, NSEC, and NSEC3 input is rejected. ANAME must be materialized before signing.

These are not footnotes hidden behind a "DNSSEC enabled" switch. They are the conditions under which an offline immutable snapshot can honestly claim to be the zone. The small-tool approach makes each condition visible, testable, and replaceable without putting private keys in the packet-serving process.

9 Zone transfer and secondary service

9.1 AXFR is a stream, not a giant datagram

AXFR transfers a complete zone over TCP. A successful stream begins with the zone's SOA, contains the zone records, and ends with the SOA again. The records may span many DNS messages. A client must continue until it sees the closing SOA under the transfer rules; reading one response is insufficient.

Transfers reveal the zone contents and can consume resources, so authorities normally restrict clients. TSIG is a common authentication mechanism in the wider ecosystem, while IP allowlists are a simpler policy with weaker identity properties.

`src/axfr.rs` provides both sides. The standalone `axfrdns` command accepts TCP only and checks client networks, loopback by default. The packaged primary also routes AXFR through `tinydns`'s existing TCP listener when `ALLOW_NETS` is set. This is required when ordinary authoritative DNS and transfers must share one address on port 53: two separate processes cannot own that TCP endpoint.

Both entry points require one AXFR question, obtain a boundary-aware transfer from `Zone`, and frame bounded messages. `Zone::transfer` excludes records beneath delegated child zones and wraps the result in the apex SOA. The integrated listener applies its transfer allow-list only to AXFR; ordinary DNS-over-TCP remains reachable by all clients allowed through the network firewall.

`axfr-get` generates a random transaction ID, validates response identity and shape, collects records until the closing SOA, renders them in tinydns source form, writes a temporary output, and atomically installs the completed file. The temporary/final path pair prevents a failed transfer from replacing usable data with a partial zone.

9.2 One request, one portable log record

The original tinydns wrote one compact record for every request. rgbdns keeps that operational contract. An IPv4 record has this shape:

```
7f000001:e214:0018 + 0001 fieldnotes.es
```

The client address, source port, DNS ID, and query type are hexadecimal. The result marker distinguishes an attempted answer (`+`), refused authority or AXFR (`-`), an unimplemented request (`I`), an unsupported class (`C`), and a malformed request (`/`). IPv6 uses the same format with a 32-hex-digit address. Query names are escaped so packet data cannot inject additional log lines.

The record goes to stderr without a timestamp. The packaged systemd service therefore sends it to journald, while a daemontools service can send the exact same stream through `multilog t` for TAI64N timestamps and file rotation. `QUERY_LOG=1` is the default. `QUERY_LOG=0` is an explicit opt-out for an installation whose traffic, retention, or privacy policy forbids full query logging.

`rgbdns-log-report` turns a bounded daily slice of that stream into an operational summary. It maps each accepted query name to the longest matching configured authoritative zone, so `www.wishful.ly` contributes to `wishful.ly` rather than becoming a separate domain. For each zone it reports the total query count and the number of distinct client addresses, then sorts by total descending. A packaged opt-in timer reads the preceding local day from journald and submits the text through a sendmail-compatible transport. Distinct addresses usually identify recursive resolvers, not individual people, and DNS queries are not HTTP pageviews. The timer belongs on one authority only when duplicate reports are unwanted.

10 The rgbdns program family

10.1 One suite, small purposes

rgbdns deliberately exposes separate commands:

Command family	Purpose
`tinydns`, `tinydns-data`, `tinydns-get`, `tinydns-edit`	authoritative service and data maintenance
`dnscache`	validating recursive resolver and cache
`axfrdns`, `axfr-get`	zone transfer server and client
`rbldns`, `rbldns-data`	address-prefix blocklist DNS
`pickdns`, `pickdns-data`	location-aware address selection
`walldns`	synthetic address/reverse answers
`dnsq`, `dnsqr`, `dnsip*`, `dnsname`, `dnsmx`, `dnstxt`	queries and diagnostics
`dnsfilter`, `dnstrace`, `random-ip`	stream lookup, delegation tracing, testing
`rgbdns-log-report`	daily per-zone query and distinct-client aggregation
`*-conf`	service-directory generation
`setuidgid`, `multilog`, `tai64n`, `tai64nlocal`	process and logging support

This composition makes privilege and failure boundaries visible. A compiler can run with write access to data while the server runs read-only. A recursive cache can be restarted without touching authority. Diagnostic clients reuse the packet and client libraries rather than embedding daemon behavior.

10.2 Specialized responders

`rbldns` treats the labels before a configured suffix as a numeric address, finds the most-specific matching IPv4 prefix in a compiled database, and returns configured A/TXT data. Parsing caps the number of numeric labels and validates networks before compilation.

`pickdns` maps client prefixes to two-byte locations and selects address sets for that location. It shuffles eligible addresses with operating-system randomness. Location-aware answers are a policy feature; clients behind shared resolvers may appear at the resolver's address, a limitation operators must understand.

`walldns` synthesizes narrowly defined forward and reverse answers without a zone database. These specialized services run through `src/special.rs`, which provides shared bounded UDP/TCP serving and passes the peer address to the handler.

The lesson is architectural: once parsing, transport, names, and record models are sound, unusual DNS policies can be small pure response functions rather than new monolithic servers.

11 Client behavior and diagnostics

11.1 A query is more than sending bytes

A DNS client creates a random transaction ID, encodes one question, sends it to an intended server, receives a response, and validates at least:

- source endpoint where the transport permits;
- transaction ID;
- QR and response shape;
- matching question;
- declared section lengths and names;
- truncation, with TCP retry when needed.

`src/client.rs` reads `DNSCACHEIP` or `/etc/resolv.conf`, supports IPv4 and IPv6 socket syntax, gets IDs from the operating system, applies UDP timeouts, rejects mismatched responses, and retries truncated UDP replies over TCP. The small command binaries format results for different use cases, while `dnsq` allows an explicit server and `dnsqr` uses recursive configuration.

`dnstrace` is conceptually different from a recursive lookup: it exposes the delegation path and intermediate authority so an operator can see where the chain stops. Good diagnosis asks four separate questions:

1. What did the stub send?
2. What did the recursive resolver cache or validate?
3. What delegation did the parent publish?
4. What does the authoritative server say directly?

Testing only the final application collapses all four layers and encourages guessing.

11.2 Practical checks

When a name fails, inspect type, server, flags, and authority section rather than asking only whether an address appeared.

```
dnsq A www.example.com 192.0.2.53
dnsq AAAA www.example.com 192.0.2.53
dnsq SOA example.com 192.0.2.53
dnstrace A www.example.com
```

Compare UDP and TCP when answers are large. Query the parent-side NS records and the child authority separately. An NXDOMAIN with an SOA is different from a timeout, SERVFAIL, or REFUSED, and each points to a different layer.

12 Security engineering in rgbdns

12.1 The packet is hostile

DNS combines nearly every parser hazard: nested lengths, compression pointers, variable counts, binary strings, recursive relationships, and network-facing availability requirements. “Written in

Rust" removes broad classes of memory corruption, but it does not automatically prevent allocation bombs, infinite loops, CPU amplification, path races, policy errors, or accepting incoherent messages.

rgbdns therefore uses several layers:

- `#![forbid(unsafe_code)]` for the library;
- explicit bounds before every wire read;
- validated `Name`, `Message`, and `RData` objects;
- limits on compression traversal, aliases, records, files, configuration lists, recursion, transfers, and cache sizes;
- cryptographic operating-system randomness for query IDs and selection;
- complete-record truncation;
- loopback-only defaults for recursion and transfer;
- atomic replacement for compiled databases and fetched zones;
- no shell interpolation when replacing a process.

Property tests in `tests/packet_properties.rs` feed arbitrary bytes to the decoder and exercise encode/decode invariants. Golden CDB fixtures compare compiled output with the expected djbdns layout. Network tests cross real UDP and TCP boundaries. Compatibility tests are valuable here because a parser can be safe yet subtly wrong, or compatible yet unsafe.

12.2 Least privilege and filesystem boundaries

The `*-conf` commands generate service directories whose run scripts execute the daemon under a selected account. rgbdns's `setuidgid` resolves the user and group, initializes supplementary groups, drops GID and UID, verifies the result, and directly replaces itself with the target program. Direct replacement preserves signals and exit status and avoids an extra shell-owned process.

Generated paths are shell-quoted and support binaries by absolute path. Configuration writers reject unsafe existing file types and apply intentional modes. CDB and AXFR update workflows install only complete outputs.

Privilege dropping is not a substitute for a restricted service account, read-only data, network policy, or supervisor hardening. It is one layer in a deployment.

13 Time and logs: TAI64N

13.1 Why a DNS suite contains time tools

Long-running daemons need logs, and djb's tools use TAI64N labels. A label has an `@`, sixteen hexadecimal digits of biased TAI seconds, and eight hexadecimal digits of nanoseconds:

```
@4000000037c219bf2ef02e94
```

TAI is a continuous atomic timescale. UTC inserts leap seconds, so converting between a POSIX/UTC timestamp and TAI requires the applicable TAI–UTC offset. The offset was 10 seconds at the Unix epoch convention used here and reached 37 seconds after the 2016 leap second.

`tai64n` timestamps each input line at the moment its first bytes are read. `tai64nlocal` recognizes a valid leading label and replaces it with local civil time at nanosecond precision. Invalid prefixes pass through unchanged. `multilog t` uses the same label generator, ensuring standalone filters and log files agree.

`src/tai64.rs` contains the complete 1972–2017 positive-leap transition table. It validates the fixed-width hexadecimal form and nanosecond range. Both stream filters use bounded memory: the localizer buffers only the 25-byte candidate prefix and streams the rest of even an extremely long line. At I/O failure the command exits 111, following the daemontools convention.

TAI64N provides sortable, unambiguous event labels. Converting to local time is a presentation step, not the archival representation.

14 Running rgbdns under supervision

14.1 The service contract

rgbdns daemons run in the foreground, emit diagnostics to standard error, take configuration from files and environment, and terminate on fatal startup errors. That is the portable contract a supervisor needs. The generated djbdns-style directories additionally provide `run` and `log/run` programs, but the daemon binaries do not require a particular supervisor.

The classic daemontools control plane is:

```
supervise service/       keep one process running
svc -u service/          bring it up
svc -d service/          bring it down
svc -t service/          send TERM
svstat service/          inspect status
```

No modern replacement is universally best. Choose according to the host and the migration boundary.

14.2 Recommendations

14.2.1 Existing Linux host: systemd

Use systemd when the machine already boots and manages services with systemd. It supplies dependency ordering, restart policy, socket and readiness models, resource controls, credential and filesystem sandboxing, a unified journal, and distribution-native administration. Avoid wrapping an rgbdns daemon in a second nested supervisor.

A minimal authoritative unit is:

```
[Unit]
Description=rgbdns authoritative DNS
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
```

```
User=dns
Group=dns
Environment=IP=192.0.2.53
Environment=PORT=53
Environment=DATA=/etc/rgbdns/data.cdb
ExecStart=/usr/local/bin/tinydns
Restart=on-failure
RestartSec=1s
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
ReadOnlyPaths=/etc/rgbdns
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE

[Install]
WantedBy=multi-user.target
```

Prefer a socket above 1024 or a narrowly bounded bind capability over running the daemon as root. Test hardening settings on the target distribution because name-service libraries and trust-anchor paths may require additional read-only access.

Command mapping:

daemontools	systemd
`svc -u service`	`systemctl start service`
`svc -d service`	`systemctl stop service`
`svc -t service`	`systemctl kill --signal=TERM service`
`svc -h service`	`systemctl kill --signal=HUP service`
`svstat service`	`systemctl status service`
`multilog` output	journal, or explicit file logging policy

14.2.2 Closest service-directory migration: runit

Use runit when you want the smallest conceptual migration from daemontools. It uses a service directory with a `run` program, keeps the supervised process in the foreground, has a companion log service, and exposes the compact `sv` control command. Existing rgbdns generated `run` scripts are close to the required shape; adjust the directory layout and enablement symlink for the distribution.

daemontools	runit
`svc -u service`	`sv up service`
`svc -d service`	`sv down service`
`svc -t service`	`sv term service`
`svstat service`	`sv status service`

Choose runit for minimal hosts, appliances, or migrations where preserving the service-directory model matters more than rich dependency and sandbox policy.

14.2.3 Strong supervision composition: s6 and s6-rc

Use s6 when precise process supervision, reliable readiness, and composable small tools are primary requirements. Its `s6-supervise` and `s6-svc` are close in spirit to `supervise` and `svc`; `s6-rc` adds declared dependencies and transactional service-state changes. The ecosystem is particularly effective in carefully constructed containers and small systems, but its compilation and directory conventions make migration more involved than runit.

daemontools	s6
`supervise service`	`s6-supervise service`
`svc -u service`	`s6-svc -u service`
`svc -d service`	`s6-svc -d service`
`svc -t service`	`s6-svc -t service`
`svstat service`	`s6-svstat service`

Choose s6/s6-rc when the team is willing to own its service database and wants more rigorous dependency transitions than ad hoc shell orchestration.

14.2.4 OpenRC and container orchestrators

On an OpenRC-based distribution, use the native init integration unless there is a deliberate reason to introduce another supervision tree. OpenRC service scripts can use its supervisor support while retaining distribution-standard boot ordering and administration.

In Kubernetes or a similar orchestrator, run one foreground rgbdns daemon per container and let the platform own restart, health, resource limits, log collection, and rollout. Use a Deployment for `tinydns` or `dnscache`, a Service for stable network reachability, readiness/liveness probes that test the intended DNS role, ConfigMaps or mounted immutable CDBs for public data, and Secrets for sensitive material. Do not put systemd, daemontools, and the orchestrator around the same single process.

An s6-based container is reasonable only when one image intentionally contains several cooperating long-lived processes and that tradeoff is explicit.

14.3 A practical selection rule

Use this order:

1. Follow the host's native manager: systemd on systemd hosts, OpenRC on OpenRC hosts.
2. For a direct service-directory replacement, choose runit.
3. For a designed supervision graph or multi-process container, choose s6/s6-rc.
4. In an orchestrated single-process container, use the orchestrator.

The least risky migration preserves one owner for restart policy and logs. Running two supervisors creates ambiguous signal paths, duplicate restarts, and status commands that disagree.

15 Operating an authoritative service

15.1 Build, stage, verify, replace

A safe publication cycle separates source editing from serving:

```
cd /etc/rgbdns
tinydns-data
tinydns-get example.com A www.example.com
```

In production, compile in a staging directory, run representative exact, wildcard, delegation, negative, IPv4, IPv6, and large-response queries, then atomically replace `data.cdb`. Retain the previous known-good database for rollback. Query the bound service over both UDP and TCP after deployment.

For an ANAME zone, test the two address families and the unaffected apex record types separately:

```
dig @192.0.2.53 example.com A +norecurse
dig @192.0.2.53 example.com AAAA +norecurse
dig @192.0.2.53 example.com SOA +norecurse
dig @192.0.2.53 example.com MX +norecurse
```

The A and AAAA answers should have the apex as their owner and should not contain a CNAME. The SOA and MX answers should come entirely from zone data. Repeat the address queries after the target changes and after its TTL expires; this verifies refresh behavior rather than only the initial lookup. Also test the chosen recursive endpoint independently, because an authoritative ANAME lookup cannot succeed when its upstream resolver is unavailable.

Do not expose the recursive service to arbitrary networks by accident. The default `ALLOW_NETS` is loopback only because an open resolver can be abused for amplification and can consume local capacity. Likewise, expand AXFR allowlists only for intended secondaries.

15.2 Full deployment walkthrough: three authoritative topologies

This walkthrough installs one editable rgbdns primary and, when selected, one rgbdns secondary and/or BuddyNS. It deliberately shares one preparation, publication, and verification path among three useful topologies:

Topology	Published authorities	AXFR readers of `a`	AXFR readers of `b`
`a + BuddyNS`	`a` and the assigned BuddyNS names	BuddyNS	not applicable
`a + b + BuddyNS`	`a`, `b`, and BuddyNS	`b` and BuddyNS	BuddyNS, if configured as an alternate master
`a + b`	`a` and `b`	`b`	none

The common path is intentionally longer than any topology-specific branch. Choose the topology once, construct the corresponding NS and AXFR lists, then reuse the same installation and verification commands.

15.2.1 Names, addresses, and the security boundary

The examples use one service zone with in-bailiwick nameservers:

```
ZONES="example.net example.org"
PRIMARY_NS=a.ns.example.net
SECONDARY_NS=b.ns.example.net
PRIMARY_PUBLIC_IP=192.0.2.53
SECONDARY_PUBLIC_IP=198.51.100.53
PRIMARY_PRIVATE_IP=10.0.1.10
SECONDARY_PRIVATE_IP=10.0.2.10
```

Replace every documentation address and name. On AWS, bind each daemon to `0.0.0.0:53`; the guest normally sees its private interface while the Internet gateway maps its Elastic IP. Use private addresses for AXFR between instances in the same VPC. Give both instances stable public addresses before publishing delegation.

Permit public UDP 53 and public TCP 53 in the cloud security group and host firewall. Ordinary DNS needs both transports, so do not limit all TCP 53 to secondaries. rgbdns applies `ALLOW_NETS` only to AXFR questions. Separately allow TCP 53 from the secondary's private address or, preferably on AWS, its security group.

The examples place `a.ns.example.net` and `b.ns.example.net` inside the served zone, so the parent needs glue for both. Some deployments use names from a separate infrastructure zone. For example, `fieldnotes.es` can use `a.ns.cron.sh` and `b.ns.cron.sh`. In that case:

- put empty address fields on the `fieldnotes.es` NS lines;
- publish the `a.ns.cron.sh` and `b.ns.cron.sh` A records in the `cron.sh` zone, not in `fieldnotes.es`;
- create glue at the parent of `cron.sh` when those names are in-bailiwick nameservers for `cron.sh`; and
- one packaged secondary instance can synchronize many zones from the same primary endpoint.

Consequently, advertise `b` only for zones included in `/var/lib/rgbdns/tinydns/zones`. Add both `fieldnotes.es` and `cron.sh` when `b` should be authoritative for both; otherwise retain other working authorities for the omitted infrastructure zone.

15.2.2 Worked deployment: `fieldnotes.es` on `a.ns.cron.sh` and `b.ns.cron.sh`

The concrete deployment uses:

Role	Name	Public address	VPC address
Debian primary	`a.ns.cron.sh`	`52.10.53.234`	`172.31.60.189`
openSUSE secondary	`b.ns.cron.sh`	`52.38.177.160`	`172.31.0.125`

The primary serves two zones from one source. `cron.sh` supplies the nameserver addresses, while `fieldnotes.es` delegates to those names plus the three assigned BuddyNS authorities:

```
# cron.sh infrastructure zone
Zcron.sh:a.ns.cron.sh:hostmaster.cron.sh:2026073001:16384:2048:1048576:2560:3600
&cron.sh:52.10.53.234:a.ns.cron.sh:3600
&cron.sh:52.38.177.160:b.ns.cron.sh:3600
&cron.sh::uz5x6wcwzfbjs8fkmkuchydn9339lf7xbxdmnp038cmyjlgg9sprr2.free.ns.buddyns.com:3600
&cron.sh::uz5dkwpjfvfwb9rh1qj93mtup0gw65s6j7vqqumch0r9gzlu8qxx39.free.ns.buddyns.com:3600
&cron.sh::uz56xw8h7fw656bpfv84pctjbl9rbzbqrw4rpzdhtvzyltpjdmx0zq.free.ns.buddyns.com:3600

# fieldnotes.es application zone
Zfieldnotes.es:a.ns.cron.sh:hostmaster.cron.sh:2026073001:16384:2048:1048576:2560:3600
&fieldnotes.es::a.ns.cron.sh:3600
&fieldnotes.es::b.ns.cron.sh:3600
&fieldnotes.es::uz5x6wcwzfbjs8fkmkuchydn9339lf7xbxdmnp038cmyjlgg9sprr2.free.ns.buddyns.com:3600
&fieldnotes.es::uz5dkwpjfvfwb9rh1qj93mtup0gw65s6j7vqqumch0r9gzlu8qxx39.free.ns.buddyns.com:3600
&fieldnotes.es::uz56xw8h7fw656bpfv84pctjbl9rbzbqrw4rpzdhtvzyltpjdmx0zq.free.ns.buddyns.com:3600
```

Increment the affected SOA serial whenever the source changes. Obtain the current BuddyNS transfer-source CIDRs from BuddyNS and store them in the login account's protected `buddyns-axfr.env`; provider networks are operational input, not constants to copy forever from a book.

From the normal `bitnami` shell on the primary, configure the role and its watched `rgbdns.data` path:

```
. "$HOME/buddyns-axfr.env"
PRIMARY_AXFR_NETS="172.31.0.125/32,$BUDDYNS_AXFR_V4"

sudo rgbdns-setup primary \
  --data "$HOME/rgbdns.data" \
  --data-drop "$HOME/rgbdns.data" \
  --data-drop-owner "$(id -un)" \
  --listen-ip 0.0.0.0 --port 53 \
  --allow-nets "$PRIMARY_AXFR_NETS" \
  --query-log 1
```

The secondary transfers both zones because it is advertised for both:

```
cron.sh
fieldnotes.es
```

From the normal openSUSE login shell:

```
. "$HOME/buddyns-axfr.env"

sudo rgbdns-setup secondary \
  --zones "cron.sh fieldnotes.es" \
  --primary 172.31.60.189 \
  --zones-drop "$HOME/rgbdns.zones" \
  --zones-drop-owner "$(id -un)" \
  --listen-ip 0.0.0.0 --port 53 \
  --allow-nets "$BUDDYNS_AXFR_V4" \
  --query-log 1
```

Use the private primary address for VPC AXFR while retaining the public addresses in delegation. Later, publish complete primary data and the desired secondary list through temporary names and atomic renames:

```
scp rgbdns.data bitnami@52.10.53.234:rgbdns.data.new
ssh bitnami@52.10.53.234 'mv rgbdns.data.new rgbdns.data'

scp rgbdns.zones "$SUSE_USER"@52.38.177.160:rgbdns.zones.new
ssh "$SUSE_USER"@52.38.177.160 'mv rgbdns.zones.new rgbdns.zones'
```

The primary importer compiles a private staged copy and leaves the live CDB unchanged on failure. The secondary importer validates, normalizes, and atomically installs the list before starting AXFR. Inspect both boundaries:

```
sudo journalctl -u rgbdns-data-import.service \
  -u rgbdns-zones-import.service \
  -u rgbdns-secondary-sync.service -n 100 --no-pager
```

The standalone `RGBDNS_SETUP.md` walkthrough contains package download and installation, AWS rules, BuddyNS, delegation, upgrades, complete verification, and troubleshooting commands for this exact deployment.

For a deployment-level ANAME transfer proof, add an undelegated reserved test zone to the primary source:

```
Zaname-
axfr.test:a.ns.cron.sh:hostmaster.cron.sh:2026073101:16384:2048:1048576:2560:3600
&aname-axfr.test::a.ns.cron.sh:3600
&aname-axfr.test::b.ns.cron.sh:3600
Aaname-axfr.test:publication.ghost.io:300
```

Publish the complete primary data atomically, add `aname-axfr.test` to the complete secondary drop list, and publish that list atomically. On the secondary, this line proves negotiated metadata survived AXFR:

```
sudo grep '^Aaname-axfr\.test' \
  /var/lib/rgbdns/tinydns/secondary-zones/aname-axfr.test.data
```

Normal queries to each public authority must return standard authoritative A or AAAA answers with no CNAME and a TTL no greater than 300. Run the standard AXFR privacy check from the secondary or another authorized VPC host:

```
dig @172.31.60.189 aname-axfr.test AXFR |
  grep -E 'TYPE65401|RGA1|publication\.ghost\.io'
```

No output is expected. The private `172.31.60.189` address is not reachable from an ordinary workstation outside AWS, and the public AXFR endpoint should remain limited to explicitly authorized source addresses.

15.2.3 Obtain and install the packages

The GitHub Actions workflows publish architecture-specific artifacts. They are not APT or Zypper repositories. Install GitHub CLI and authenticate on a machine allowed to retrieve the artifacts:

```
gh auth login
```

Select the newest successful Debian build without depending on the version-specific `--status` option found only in newer GitHub CLI releases:

```
DEB_RUN_ID=$(
  gh run list -R querygraph/rgbdns \
    -w build-deb.yml -b master -L 50 \
    --json databaseId,conclusion \
    --jq '.[] | select(.conclusion == "success") | .databaseId' |
  head -n 1
)
mkdir -p "$HOME/rgbdns-deb"
gh run download "$DEB_RUN_ID" \
  -R querygraph/rgbdns \
  -n rgbdns-debian-amd64 \
  -D "$HOME/rgbdns-deb"
```

On the Debian or Ubuntu primary, install the downloaded package:

```
sudo apt install "$HOME"/rgbdns-deb/rgbdns_*_amd64.deb
sudo dpkg --audit
dpkg-query -W -f='${Status} ${Version}\n' rgbdns
```

The package intentionally replaces Debian's djbdns and daemontools command packages because both suites own paths such as `/usr/bin/tinydns-get` and `/usr/bin/multilog`. Review APT's removal plan before confirming on a host that already runs those services. Never use `dpkg --force-overwrite`.

For a selected topology containing `b`, download the newest successful openSUSE RPM artifact on the Leap 16 secondary:

```
RPM_RUN_ID=$(
  gh run list -R querygraph/rgbdns \
    -w build-rpm.yml -b master -L 50 \
    --json databaseId,conclusion \
    --jq '.[] | select(.conclusion == "success") | .databaseId' |
  head -n 1
)
mkdir -p "$HOME/rgbdns-rpm"
gh run download "$RPM_RUN_ID" \
  -R querygraph/rgbdns \
  -n rgbdns-opensuse-leap16-x86_64 \
  -D "$HOME/rgbdns-rpm"
RPM=$(find "$HOME/rgbdns-rpm/RPMS/x86_64" \
  -maxdepth 1 -name 'rgbdns-[0-9]*.x86_64.rpm' -print -quit)
rpm -K "$RPM"
sudo zypper --non-interactive --no-gpg-checks install "$RPM"
sudo rpm -V rgbdns
```

The artifact retains its `RPMS/x86_64` and `SRPMS` directories. Install the binary package under `RPMS/x86_64`; `SRPMS` contains the source RPM. The development package is payload-verified but not repository-signed, hence the explicit `--no-gpg-checks`. The RPM obsoletes and conflicts with RPM packages named `djbdns` and `daemontools`; inspect Zypper's transaction if either is installed.

Both packages create the non-login `rgbdns` user and group, protected configuration under `/etc/rgbdns`, state under `/var/lib/rgbdns/tinydns`, and hardened systemd units. Installation does not publish placeholder DNS data or enable authority.

Verify the installed account and units:

```
getent passwd rgbdns
getent group rgbdns
systemctl list-unit-files 'rgbdns-*'
```

15.2.4 Build one primary source file

Start with common application records and one SOA serial. A sortable `YYYYMMDDNN` serial is convenient; increment it before every publication. Construct the authoritative NS portion from exactly one topology block below.

Common records:

```
Zexample.net:a.ns.example.net:hostmaster.example.net:2026072901:16384:2048:1048576:2560:3600
+example.net:192.0.2.80:3600
C*.example.net:example.net:3600
```

For `a + BuddyNS`:

```
&example.net:192.0.2.53:a.ns.example.net:3600
&example.net::<BuddyNS name 1>:3600
&example.net::<BuddyNS name 2>:3600
&example.net::<BuddyNS name 3>:3600
```

For `a + b + BuddyNS`:

```
&example.net:192.0.2.53:a.ns.example.net:3600
&example.net:198.51.100.53:b.ns.example.net:3600
&example.net::<BuddyNS name 1>:3600
&example.net::<BuddyNS name 2>:3600
&example.net::<BuddyNS name 3>:3600
```

For `a + b`:

```
&example.net:192.0.2.53:a.ns.example.net:3600
&example.net:198.51.100.53:b.ns.example.net:3600
```

An & line with an address creates the NS record and its address/glue. The empty address fields on BuddyNS lines are intentional: those names belong to BuddyNS. Replace the account-specific placeholders with the exact names shown in BuddyBoard.

Store the assembled source as `/root/rgbdns.data` on the primary. Protect and compile a disposable copy before changing the service:

```
sudo install -o root -g root -m 0600 rgbdns.data /root/rgbdns.data
check_dir=$(mktemp -d)
sudo install -o "$(id -u)" -g "$(id -g)" -m 0600 \
  /root/rgbdns.data "$check_dir/data"
(cd "$check_dir" && tinydns-data)
ls -lh "$check_dir/data.cdb"
rm -r "$check_dir"
```

Compilation proves syntax and semantic consistency, not that the chosen addresses, delegation, mail policy, or application records are correct. Compare the new source with the old zone before cutover. AXFR from an existing authority is the best inventory when allowed; otherwise query all known record types and names from configuration management.

15.2.5 Construct the AXFR allow-list once

For a topology containing BuddyNS, copy BuddyBoard's current published transfer-source addresses into a protected, sourceable file. Express individual IPv4 sources as `/32` networks:

```
sudo install -o root -g root -m 0600 \
  buddyns-axfr.env /etc/rgbdns/buddyns-axfr.env
. /etc/rgbdns/buddyns-axfr.env
```

The file has this form:

```
BUDDYNS_AXFR_V4='203.0.113.10/32,203.0.113.11/32'
```

Treat the addresses as provider-maintained data, not constants copied forever from a book. Reconcile the file with BuddyBoard before a deployment and after provider network changes.

Choose the primary allow-list:

```
# a + BuddyNS
PRIMARY_ALLOW_NETS=$BUDDYNS_AXFR_V4
```

```
# a + b + BuddyNS
PRIMARY_ALLOW_NETS="$SECONDARY_PRIVATE_IP/32,$BUDDYNS_AXFR_V4"

# a + b
PRIMARY_ALLOW_NETS="$SECONDARY_PRIVATE_IP/32"
```

Run only the assignment for the selected topology. Never allow the entire VPC when one stable secondary address or security-group path is sufficient.

15.2.6 Configure and cut over the primary

First stage configuration without claiming port 53:

```
sudo rgbdns-setup primary \
  --data /root/rgbdns.data \
  --listen-ip 0.0.0.0 \
  --port 53 \
  --allow-nets "$PRIMARY_ALLOW_NETS" \
  --no-start
sudo -u rgbdns /usr/lib/rgbdns/compile-zone
sudo systemd-analyze verify \
  /lib/systemd/system/rgbdns-tinydns.service 2>/dev/null ||
sudo systemd-analyze verify \
  /usr/lib/systemd/system/rgbdns-tinydns.service
sudo ls -lah /var/lib/rgbdns/tinydns
sudo cat /etc/rgbdns/tinydns.env
```

Setup watches `rgbdns.data` in the invoking sudo user's home directory. After editing the complete source locally, publish it atomically:

```
scp rgbdns.data a.ns.example.net:rgbdns.data.new
ssh a.ns.example.net 'mv rgbdns.data.new rgbdns.data'
```

`rgbdns-data.path` verifies ownership, compiles a private staged copy, and replaces the live source and CDB only after successful compilation. Invalid, partial, or symlinked uploads leave the currently served database unchanged. Use `--data-drop FILE` and `--data-drop-owner USER` when another destination is required.

The two unit paths cover Debian-family and openSUSE layouts. An unrelated legacy-unit warning from `systemd-analyze` does not invalidate a successful rgbdns unit check.

On a migration host, identify the existing owner of port 53:

```
sudo ss -lntup '( sport = :53 )'
```

Stop only the old authoritative services. Do not stop an entire `runsvdir` tree on a host where it also owns unrelated applications. For a classic djbdns layout:

```
sudo sv down /etc/axfrdns /etc/axfrdns/log
sudo sv down /etc/tinydns /etc/tinydns/log
sudo ss -lntup '( sport = :53 )'
```

Then enable rgbdns:

```
sudo systemctl enable --now rgbdns-tinydns.service
sudo systemctl status rgbdns-tinydns.service --no-pager --full
sudo ss -lntup '( sport = :53 )'
```

The authoritative daemon serves normal UDP, normal TCP, and allowed AXFR on the same port. Do not start the separately packaged `axfrdns` compatibility command on port 53.

Verify the primary locally and publicly before configuring delegation:

```
dig @127.0.0.1 example.net SOA +norecurse
dig @127.0.0.1 example.net NS +norecurse
dig @127.0.0.1 example.net A +norecurse
dig +tcp @127.0.0.1 example.net SOA +norecurse

dig @192.0.2.53 example.net SOA +norecurse
dig @192.0.2.53 example.net NS +norecurse
dig +tcp @192.0.2.53 example.net SOA +norecurse
```

Require `status: NOERROR`, the `aa` flag, the intended serial, the complete NS set, and correct address records.

15.2.7 Configure `b` when the topology includes it

The primary must allow the secondary's source address, and the network path must permit TCP 53, before this step. On the openSUSE secondary:

```
sudo ss -lntup '( sport = :53 )'
sudo rgbdns-setup secondary \
  --zone example.net \
  --zone example.org \
  --primary 10.0.1.10 \
  --listen-ip 0.0.0.0
```

For `a + b + BuddyNS`, BuddyNS may read from `b` as an alternate master. In that case load the same current provider list and pass it to the secondary:

```
. /etc/rgbdns/buddyns-axfr.env
sudo rgbdns-setup secondary \
  --zones "example.net example.org" \
  --primary 10.0.1.10 \
  --listen-ip 0.0.0.0 \
  --allow-nets "$BUDDYNS_AXFR_V4"
```

Choose one of those two setup commands. `--zone` is repeatable; `--zones` accepts a whitespace- or comma-separated list. Setup performs a complete AXFR for each zone, validates every response and its SOA bookends, and requires one valid snapshot per zone before initially starting authority. On later runs, a failed refresh retains that zone's last-known-good snapshot while successful zones advance. The snapshots are compiled together and the combined CDB is installed atomically. The randomized five-minute timer does not use NOTIFY or IXFR.

Setup stores the canonical one-zone-per-line list in `/var/lib/rgbdns/tinydns/zones` and watches `rgbdns.zones` in the invoking sudo user's home. Manage later changes as a file and publish them with an atomic rename:

```
scp rgbdns.zones b.ns.example.net:rgbdns.zones.new
ssh b.ns.example.net 'mv rgbdns.zones.new rgbdns.zones'
```

`rgbdns-zones.path` validates ownership and contents before replacing the canonical list and starting synchronization. Invalid, empty, symlinked, or partially uploaded lists leave the active configuration unchanged. Use `--zones-drop FILE` and `--zones-drop-owner USER` during setup when the default home-directory destination is not appropriate.

Check the one-shot synchronization result:

```
systemctl show rgbdns-secondary-sync.service \
  -p Result -p ExecMainStatus -p ActiveState -p SubState
```

A successful completed run reads:

```
Result=success
ExecMainStatus=0
ActiveState=inactive
SubState=dead
```

`inactive/dead` is correct for a finished `Type=oneshot` service. The `/run/rgbdns` runtime directory and its lock exist only while synchronization runs; systemd removes and recreates them for each invocation.

Verify service, timer, and answers:

```
sudo systemctl enable --now rgbdns-tinydns.service
sudo systemctl enable --now rgbdns-secondary-sync.timer
sudo systemctl enable --now rgbdns-zones.path
systemctl list-timers rgbdns-secondary-sync.timer
systemctl status rgbdns-zones.path
sudo ss -lntup '( sport = :53 )'
dig @127.0.0.1 example.net SOA +norecurse
dig @198.51.100.53 example.net SOA +norecurse
dig +tcp @198.51.100.53 example.net SOA +norecurse
```

Every secondary serial must match its primary counterpart. To force a refresh after a publication:

```
sudo systemctl start rgbdns-secondary-sync.service
sudo journalctl -u rgbdns-secondary-sync.service -n 50 --no-pager
```

15.2.8 Configure BuddyNS when the topology includes it

In BuddyBoard:

1. add the zone;
2. configure `192.0.2.53:53` as a transfer master;
3. for `a + b + BuddyNS`, optionally add `198.51.100.53:53` as another master;
4. require the provider's transfer test to succeed; and

5. record the exact assigned BuddyNS names and transfer-source addresses.

The source zone's BuddyNS NS records, BuddyBoard's assigned names, and the eventual parent delegation must agree exactly. A provider transfer test should succeed before any registrar change.

From an allowed transfer source, or with a controlled temporary test address added to `ALLOW_NETS`, verify:

```
dig +tcp AXFR example.net @192.0.2.53
dig +tcp AXFR example.net @198.51.100.53  # when b permits BuddyNS
```

An unlisted client should receive `REFUSED`. Do not broaden the allow-list merely to make an arbitrary workstation AXFR test succeed.

15.2.9 Publish glue and delegation last

Create or verify registrar host objects before adding in-bailiwick nameservers:

```
a.ns.example.net = 192.0.2.53
b.ns.example.net = 198.51.100.53   # topologies containing b
```

Then publish the parent delegation matching the selected topology:

- `a + BuddyNS`: `a` plus the assigned BuddyNS names;
- `a + b + BuddyNS`: `a`, `b`, plus the assigned BuddyNS names;
- `a + b`: `a` and `b`.

Do not advertise `b` before it answers the current serial publicly. During a migration, keep old working secondaries in both the child NS RRset and parent delegation until new authorities pass UDP, TCP, SOA, and negative-answer tests. Remove old secondaries in a later serial change after parent updates have propagated.

Trace the parent and query every authority:

```
dig +trace +nodnssec example.net NS
dig @192.0.2.53 example.net SOA +norecurse
dig @198.51.100.53 example.net SOA +norecurse
```

Query each BuddyNS hostname as well when selected. Compare serials, NS RRsets, and authoritative flags. Also test a known name, a nonexistent name, UDP, and TCP:

```
dig @192.0.2.53 www.example.net A +norecurse
dig @192.0.2.53 does-not-exist.example.net A +norecurse
dig +tcp @192.0.2.53 www.example.net A +norecurse
```

15.2.10 Publish changes, upgrade, and recover

For each zone change:

1. edit the protected canonical source;
2. increment the affected SOA serial;
3. compile a disposable copy;
4. rerun `rgbdns-setup primary` with the complete allow-list;
5. query the primary;

6. force or await secondary refresh; and
7. compare every authority's serial.

Package upgrades preserve configuration and state. Upgrade a downloaded Debian package with `apt install /path/package.deb` and an RPM with:

```
sudo zypper --non-interactive --no-gpg-checks install \
  /path/to/rgbdns-VERSION-RELEASE.x86_64.rpm
sudo rpm -V rgbdns
```

A fresh package installation deliberately starts no role-specific automation. Run `rgbdns-setup primary` or `rgbdns-setup  secondary` once. Beginning with 0.3.3, upgrades inspect the role already recorded under `/etc/rgbdns`:

- a primary with `data-drop.env` has authority and `rgbdns-data.path` restored;
- a secondary has authority and `rgbdns-secondary-sync.timer` restored and, when `zones-drop.env` exists, `rgbdns-zones.path` restored; and
- an unconfigured installation remains inactive.

Verify after an upgrade that `rgbdns-tinydns` is active and that a primary picker is `active (waiting)`, or that both the secondary timer and zone picker are `active (waiting)`.

To repurpose a host, prepare the complete input for the destination role and run the opposite `rgbdns-setup` command. A primary-to-secondary conversion requires a valid zone list and successful initial AXFR. A secondary-to-primary conversion requires a complete, validated data file. The setup command removes the old role configuration, disables its picker and timer, and activates only the new role. Do not convert roles by manually enabling both sets of units.

If secondary setup fails, it deliberately leaves authority stopped until the first valid transfer completes. Diagnose in this order:

```
sudo systemctl status rgbdns-secondary-sync.service --no-pager --full
sudo journalctl -u rgbdns-secondary-sync.service -n 100 --no-pager
dig +tcp @10.0.1.10 example.net SOA +norecurse
sudo cat /etc/rgbdns/secondary.env
sudo cat /var/lib/rgbdns/tinydns/zones
sudo cat /etc/rgbdns/tinydns.env
```

A connection timeout points toward routes, security groups, or firewalls. `REFUSED` points toward `ALLOW_NETS` or an unexpected NAT source address. A validation error points toward the transferred zone. A successful one-shot with a refused local query means `rgbdns-tinydns.service` is not yet active.

After every reboot or upgrade, verify:

```
systemctl is-enabled rgbdns-tinydns.service
systemctl is-active rgbdns-tinydns.service
systemctl list-timers rgbdns-secondary-sync.timer
sudo ss -lntup '( sport = :53 )'
```

Monitor public UDP and TCP answers, unit restarts, timer failures, SOA convergence, transfer failures, and disk space. Keep the editable primary source in protected configuration management or backup. Secondary DNS improves serving availability; it is not a backup of the canonical source.

The distribution-specific deployment guides contain additional AWS, firewall, SELinux, and troubleshooting detail: `docs/DEBIAN.md` and `docs/OPENSUSE.md`.

15.3 Observe the right signals

Useful signals include:

- query and error rate by transport;
- truncated UDP responses and TCP retries;
- SERVFAIL, REFUSED, NXDOMAIN, and validation-failure rates;
- resolver cache capacity and latency percentiles;
- process restarts and file-descriptor use;
- root-hint and trust-anchor freshness;
- ANAME refresh latency, upstream failures, cache misses, and synthesized TTLs;
- time synchronization;
- CDB build identity and deployment time.

High NXDOMAIN volume is not automatically an incident; browsers, typo traffic, and discovery protocols generate it. A change from baseline paired with latency or SERVFAIL is more meaningful.

TAI64N log labels make events stable for storage. Convert them for human display at the edge:

```
tail -f main/current | tai64nlocal
```

16 Testing DNS software

16.1 Layers of evidence

Unit tests establish local invariants: name limits, record parsing, lookup outcomes, leap conversion. Property tests explore parser state spaces that examples miss. Golden fixtures establish compatibility with an external file format. Integration tests cross process and socket boundaries. Live interoperability tests compare behavior with independent clients and servers.

rgbdns uses all of these. A useful local sequence is:

```
cargo fmt --check
cargo test
cargo clippy --all-targets --all-features -- -D warnings
```

Network tests bind unprivileged loopback ports. CDB tests compile canonical fixtures and compare entries. Daemontools tests exercise process replacement, rotation, and TAI64 filter behavior. Packet properties assert that arbitrary input does not panic and that supported structured messages survive encode/decode round trips.

16.2 Adversarial cases worth keeping

Every DNS implementation should retain regression cases for:

- a compression pointer to itself or a pointer cycle;
- a pointer or RDATA length just beyond the packet;
- maximum-length labels and names;
- counts that cannot be satisfied by the remaining bytes;
- duplicate or malformed OPT records;
- tiny advertised transport limits;
- CNAME loops and excessive chains;
- ANAME self-reference, upstream CNAME loops, excessive address results, and resolver failure;
- wildcard names blocked by existing nodes;
- delegation cuts beneath an authoritative apex;
- NODATA versus NXDOMAIN;
- AXFR without a closing SOA;
- an enormous log line;
- configuration counts at and beyond each bound.

Tests should assert protocol meaning, not only that the process remains alive. A safe FORMERR is better than a crash, but a silent NOERROR can still be a serious bug.

16.3 Conformance as an executable specification

The conformance suite turns protocol prose into named, reviewable cases. Its scope is the DNS surface rgbdns implements; it does not imply support for every extension ever assigned by IANA. The principal coverage is:

Standard	Behavior exercised
RFC 1035	header identity, flags, names, compression, typed RDATA, UDP and TCP results
RFC 2181	in-bailiwick glue, coherent RRset TTLs, duplicate suppression, CNAME exclusivity
RFC 2308	NXDOMAIN versus NODATA, authoritative SOA, negative-cache TTL
RFC 3597	unknown QTYPE behavior and lossless opaque RDATA
RFC 4343	case-insensitive name identity with query-case preservation
RFC 4592	closest-encloser wildcard synthesis and empty non-terminals
RFC 5936	AXFR framing, identity, flags, SOA bookends, and zone boundaries
RFC 6891	one root-owned OPT, payload negotiation, DO, BADVERS, and unknown options
RFC 7766	TCP framing, connection reuse, pipelining, and full-size responses
RFC 8906	the authoritative-server matrix for unknown types, opcodes, flags, and EDNS fields
RFC 9619	exactly one question in a standard query

This is more useful than a single "RFC compliant" label. A test name identifies the rule, a packet fixture demonstrates it, and a failure points to a specific semantic regression.

RFC 8906 is especially valuable because it tests how a server behaves at the edges of what it understands. An unknown ordinary type is not a protocol error: the answer depends on whether the owner name exists. An unknown opcode is different. Because that opcode may define a body layout unlike QUERY, the server must produce NOTIMP from the header without first interpreting the body as an ordinary question. Unknown EDNS options are structurally validated and then ignored. An unsupported EDNS version produces BADVERS while retaining an OPT response.

The independent `drill` integration test supplies another boundary. It launches the real `tinydns` binary and asks the ldns client to make UDP, TCP, EDNS, mixed-case, and unknown-type queries. This catches accidental agreement between rgbdns's own encoder and decoder: the request and response cross an implementation boundary.

The complete focused matrix is:

```
cargo test --test rfc_conformance
cargo test --test wire_security
cargo test --test packet_properties
cargo test --test drill_interop
```

The generated suite exercises forty thousand cases per complete run. It feeds arbitrary bytes to the decoder, reparses every accepted packet, generates structured messages for semantic round trips, and changes ASCII letter case without changing DNS name identity. A separate truncation corpus tries every prefix of a valid structured packet. These properties do not prove the absence of all parser defects, but they explore combinations that hand-written examples rarely anticipate.

16.4 Hardening found by conformance work

Conformance testing improved the implementation rather than merely describing it.

The name decoder now records valid prior name boundaries. A compression pointer must be backward *and* must target one of those boundaries. Merely pointing at earlier bytes that happen to resemble a label sequence is rejected. This closes a class of ambiguous parses without forbidding legal compression.

Stub responses are bound to the request ID, QR bit, opcode, and exact question. TCP responses carrying TC are rejected. AXFR applies the same identity checks and additionally requires authoritative, non-truncated messages, controlled question repetition, an empty authority section, matching opening and closing SOAs, and records confined to the requested zone. These rules prevent a plausible-looking but unrelated response from being accepted as the answer to the outstanding operation.

Zone loading rejects a CNAME owner that also has other data and rejects multiple different CNAME targets. Before transmission, RRsets are normalized to their minimum TTL and duplicate records are removed. Negative answers cap the SOA TTL at the SOA MINIMUM field as RFC 2308 requires. EDNS OPT records in the wrong section and duplicate OPT records produce FORMERR.

The UDP and TCP daemons now share one bounded transport module. TCP connections carry deadlines, use a fixed worker pool, accept multiple framed queries, and support pipelined requests. This removes duplicated socket code while making RFC 7766 behavior an invariant shared by the authoritative and specialized servers.

16.5 Benchmarks and evidence-driven optimization

Correctness gates run before performance conclusions. The benchmark is a dependency-free stable-Rust harness in `benches/dns_core.rs`; the same harness is available as `examples/dns_core_bench.rs` for quick release-mode runs:

```
cargo bench --bench dns_core
RGBDNS_BENCH_ITERATIONS=10000 \
  cargo run --release --example dns_core_bench
```

It warms every operation, passes values through `std::hint::black_box`, and reports nanoseconds per operation. Measurements are comparable only on the same host, toolchain, power state, and iteration count. Wire size is reported beside CPU time because DNS compression exchanges encoder work for fewer network bytes.

The July 2026 checkpoint used release mode on one aarch64 Android host:

Operation	Baseline	Optimized	Result
Encoded 64-record response	2,147 bytes	1,059 bytes	50.7% smaller
Decode small query	542 ns	458 ns	15.5% faster
Decode 64-record response	52,661 ns	29,540 ns	43.9% faster
Encode 64-record response	2,318 ns	5,309 ns	2.3 times slower
Exact lookup, 1,000 names	1,262 ns	1,244 ns	1.4% faster
NXDOMAIN, 1,000 names	29,889 ns	2,726 ns	11.0 times faster
Small authoritative response	17,007 ns	7,714 ns	54.6% faster
Truncate 200-record response	3,098,232 ns	2,570,077 ns	17.0% faster

Three structural changes explain most of the gains.

First, `Zone` maintains an index of every node, including empty non-terminals. A clearly absent name can return NXDOMAIN without scanning the records of a thousand-name zone. Conditional records still take the visibility path, so the index does not erase time or location semantics.

Second, response truncation searches the number of tail records to remove instead of encoding once for every removed record. It preserves the question and OPT record as long as possible and validates the final candidate against the transport limit.

Third, the packet writer records complete names and suffixes for RFC 1035 compression. RRsets tend to repeat the immediately preceding owner, so a last-owner cache avoids rebuilding and hashing suffix keys on the dominant path. The first compression design encoded the 64-record case in 34,075 ns; the cache reduced that to 5,309 ns.

The remaining encoder regression is intentional and visible. Compression makes the example packet roughly half as large while taking more local CPU than the old uncompressed writer. That is a defensible trade for an authoritative server because it reduces datagram pressure, TCP bytes, and downstream decode work. Recording the regression matters: optimization should reveal tradeoffs, not hide them behind one favorable number.

17 Reading the rgbdns source

17.1 A path through the code

Read the project in dependency order:

1. `src/name.rs` — the foundational name invariant.
2. `src/packet.rs` — types and bounded wire codec.
3. `src/zone.rs` — tinydns source and authoritative lookup semantics.
4. `src/cdb.rs` — compiled compatibility format.
5. `src/server.rs` — query validation, answer construction, transport limits.
6. `src/client.rs` — stub behavior and TCP fallback.
7. `src/axfr.rs` — streaming zone movement and atomic installation.
8. `src/dnscache_config.rs` and `src/bin/dnscache.rs` — iterative recursion, DNSSEC, forwarding, access, and resource policy.
9. `src/rbl.rs`, `src/pick.rs`, `src/wall.rs`, and `src/special.rs` — specialized responders.
10. `src/conf.rs`, `src/setuidgid.rs`, `src/multilog.rs`, and `src/tai64.rs` — deployment and operations.

The binaries in `src/bin` should then look thin. That is intentional. They parse the command contract, load configuration, call a library boundary, print diagnostics, and map fatal errors to the suite's exit convention.

17.2 Design patterns to carry elsewhere

Several rgbdns choices generalize beyond DNS.

Parse into valid types. If an invalid name can circulate as an ordinary string, every consumer must rediscover validation.

Bound dimensions independently. A packet byte limit does not replace a compression-depth limit; a cache byte limit does not replace a recursion-depth limit.

Separate policy from mechanism. `transport.rs` owns bounded UDP and TCP mechanics while the authoritative and specialized handlers own answer policy.

Compile mutable source into immutable serving data. This gives validation, atomic rollout, simple readers, and easy rollback.

Preserve protocol distinctions internally. A `Lookup` enum prevents NXDOMAIN, NODATA, referral, and refusal from collapsing into "no records."

Run in the foreground. It composes with old and new supervisors and keeps signals understandable.

Treat compatibility files as hostile. Historical layout fidelity need not mean historical trust assumptions.

Part II: Codebase exploration

The first part built DNS from its protocol obligations. This part reads rgbdns as a Rust system. Each chapter follows one execution path, names the abstraction that carries the obligation, and links directly to the implementation. The Obsidian edition adds live fragment cards beside these links: a reader can move from a claim to the exact symbol, then outward to the complete collocated source file.

18 Rust as a protocol-design tool

The important comparison with the original C is not that Rust uses newer syntax. It is that rgbdns can express DNS invariants at boundaries and have the compiler preserve them across the program.

The crate begins with `#![forbid(unsafe_code)]` in `src/lib.rs`. This is stronger than merely having no currently known unsafe block: a later contribution cannot introduce one without first making an explicit, reviewable change to the crate policy. The implementation still performs byte-level packet parsing, binary CDB loading, socket I/O, process credential changes, and concurrency. Those jobs do not require unchecked pointer arithmetic.

Rust improves the design along four axes.

Ownership makes lifetime and aliasing rules executable. A decoded `Message` owns its questions and records. A server borrows a `Zone` while constructing a response. Shared handlers use `Arc`, making cross-thread ownership visible in the type rather than implicit in process convention. There is no corresponding path for a response to retain a dangling pointer into the query buffer.

Algebraic data types preserve protocol distinctions. `RecordType` retains unknown numeric types as `Unknown(u16)`. `RData` distinguishes addresses, names, MX, SOA, TXT, SRV, CAA, EDNS, and opaque future data. `Lookup` distinguishes an answer, referral, NODATA, NXDOMAIN, and refusal. In C these states are commonly represented by related integers, nullable pointers, and out-parameters. In Rust, a `match` makes the cases locally exhaustive.

Fallible work is visible. Parsing and I/O return `Result`; optional facts return `Option`. The `?` operator propagates a typed failure without the unchecked sentinel values that make C error paths easy to omit. Conversion such as `u16::try_from(response.len())` documents the narrowing boundary and rejects overflow.

Concurrency composes with ownership. The bounded transport layer shares an immutable handler through `Arc<Handler>` and gives each TCP connection an exclusive mutable `TcpStream`. The compiler prevents a worker from retaining a borrow of a stack packet buffer after the buffer is reused.

Rust does not prove the DNS protocol correct. It removes broad classes of memory corruption and turns many design assumptions into compile-time or construction-time obligations. The remaining protocol work becomes visible enough to test directly.

19 Valid names instead of hopeful strings

`Name` is the first load-bearing abstraction. It stores a sequence of byte labels rather than a UTF-8 domain string. Its constructor is private to the module; all construction passes through parsing or `from_labels`, and both reach the same validation rule:

```
fn validate(labels: &[Vec<u8>]) -> Result<()> {
    if labels.iter().any(|l| l.is_empty() || l.len() > 63) {
        return Err(Error::InvalidName(
            "label must contain 1..=63 octets".into(),
        ));
    }
    let len = 1 + labels.iter().map(|l| l.len() + 1).sum::<usize>();
    if len > 255 {
        return Err(Error::InvalidName("wire name exceeds 255 octets".into()));
    }
    Ok(())
}
```

That small private function changes the rest of the codebase. `Zone` can use `Name` as a `BTreeMap` key without rechecking label lengths. The packet writer can calculate `wire_len` without wondering whether it will overflow the DNS name limit. `parent`, `suffix`, `wildcard`, and `is_subdomain_of` operate on labels rather than fragile dotted-string suffixes.

DNS identity is case-insensitive but responses should preserve the query's case. `Name` therefore retains original bytes while implementing `Eq`, `Hash`, and `Ord` with ASCII-folded comparisons. In C this invariant must be remembered by every hash table and comparison call site. Here it belongs to the key type.

This is a zero-surprise form of abstraction. It adds allocations when a name is built, but it removes repeated parsing and validation later. The benchmarked hot paths operate on already validated values, while the network boundary absorbs the cost once.

20 A bounded wire codec

DNS packets are attacker-controlled binary graphs: compression pointers can jump backward, names can share suffixes, section counts can lie, and RDATA lengths can disagree with actual bytes. `packet.rs` contains the codec and deliberately keeps the reader state small:

```
struct Reader<'a> {
    b: &'a [u8],
    p: usize,
    name_offsets: Vec<bool>,
}
```

The lifetime on `b` prevents the reader from outliving its input. Every scalar read uses slice bounds checks. Name decoding separately limits pointer hops, requires pointers to move backward, and records valid prior name boundaries. The last rule is stricter than merely checking that a pointer lands inside the packet: an interior byte can accidentally look like a valid label.

Decoded records become an `RData` variant. Unknown types are not discarded; they become opaque bytes paired with their numeric `RecordType`. This is the extension-safe behavior required by modern DNS. It also means an encoder can round-trip data it does not understand.

The writer uses compression, but optimization remains subordinate to a valid message. A last-owner cache handles repeated owners cheaply while suffix sharing reduces wire size across related names. The July 2026 benchmark shows the trade: a 64-record answer fell from 2,147 to 1,059 bytes, while compression made encoding slower than the uncompressed baseline. On DNS, fewer datagrams and less amplification surface can be worth several microseconds of local CPU.

Truncation uses a bounded search for the largest response that fits instead of repeatedly rebuilding one record at a time. The result preserves complete RRsets and required EDNS state. Performance work is therefore expressed as an algorithmic improvement behind the same `Message::encode` contract.

21 Zone data as an indexed semantic model

`Zone` is more than a parser for tinydns text. It is the semantic index used by authoritative answers:

```
pub struct Zone {
    records: BTreeMap<Name, Vec<Record>>,
    metadata: BTreeMap<Name, Vec<RecordMetadata>>,
    authoritative: BTreeSet<Name>,
    delegations: BTreeSet<Name>,
    locations: Vec<(Vec<u8>, [u8; 2])>,
    current_metadata: RecordMetadata,
    default_serial: u32,
    nodes: BTreeSet<Name>,
    unqualified_nodes: BTreeSet<Name>,
}
```

The maps hold records and djbdns location metadata. The sets encode facts that would otherwise require scans: zone apexes, delegation cuts, all existing nodes, and nodes that exist independently of location-qualified records. This all-node index is why the optimized NXDOMAIN benchmark is about eleven times faster than the earlier scan-based implementation.

The type also prevents semantic drift. Parsing validates CNAME exclusivity once. Lookup returns a `Lookup` enum, so absence cannot collapse into one null result:

- `Answer` carries the matching RRset.
- `Referral` carries delegation NS records and in-bailiwick glue.
- `NoData` says the name exists but the requested type does not.
- `NxDomain` says the name itself does not exist.
- `Refused` says the server is not authoritative for the question.

Wildcard processing uses the `nodes` index to find the closest encloser. Delegation processing uses the explicit `delegations` set. Transfer processing walks the same model while excluding child-zone contents. One representation therefore supplies ordinary answers, negative answers, wildcards, referrals, and AXFR without five subtly different interpretations of a zone.

22 From query bytes to an authoritative answer

`server::respond` is the central authoritative pipeline. Its shape is intentionally linear:

1. Reject an unknown opcode from the header without misparsing its body as a standard query.
2. Decode the packet, mapping malformed standard queries to `FORMERR`.
3. Enforce one question and valid OPT placement.
4. Derive the UDP response limit from EDNS and the transport ceiling.
5. Ask `Zone` for a typed `Lookup`.
6. Expand bounded CNAME chains and add relevant target addresses.
7. Normalize RRset TTLs and remove duplicates.
8. Encode or truncate the response.

The code separates mechanism from policy. `transport.rs` knows UDP datagrams, TCP length prefixes, timeouts, persistent connections, and a fixed worker bound. It knows nothing about zones. The handler knows DNS policy but receives transport limits and client identity as ordinary parameters. That separation lets specialized services reuse the network machinery without pretending to be authoritative zones.

The original djbdns family achieved robustness partly through small processes. rgbdns retains that decomposition while strengthening in-process boundaries. The binaries under `src/bin` are mostly adapters: environment, configuration, a library call, and the djbdns-compatible fatal exit convention. Small executables remain independently supervisable, but common logic is testable as ordinary Rust functions.

23 CDB compatibility without trusting the file

Compatibility is most valuable at the data boundary. rgbdns reads and writes the original tinydns `data.cdb` layout, so operators can preserve compilation and rollout habits. `cdb.rs` does not, however, inherit the old assumption that the compiled file is trustworthy.

The loader applies independent limits and checked arithmetic:

- the complete database is capped at one GiB;
- the 2,048-byte CDB header must exist;
- every hash-table position and slot count must fit inside the file;
- key and value lengths use `checked_add`;
- markers, locations, names, TTLs, cutoffs, and type-specific RDATA are validated before a `Record` enters a `Zone`.

This is a useful modernization pattern: preserve a durable external format, replace its implicit in-memory trust model. Operators gain compatibility; the serving process receives validated Rust values rather than pointers into a memory-mapped byte region.

Compilation likewise crosses an explicit boundary. A parsed `Zone` is written to a temporary CDB and then installed through the command workflow. Serving data is immutable between deployments. Rust's ownership does not itself make the rollout atomic, but it makes the stages—source text, validated model, compiled bytes, installed file—unambiguous.

24 Recursion by composition

Authoritative DNS is implemented in rgbdns's own small model. Recursive DNS, DNSSEC validation, caching, and upstream transport are composed from Hickory in `src/bin/dnscache.rs`. This is not a retreat from the rewrite; it is a deliberate abstraction boundary.

rgbdns owns policy that must remain djbdns-compatible or operator-visible: root hints, forwarding zones, allowed networks, cache budgets, recursion limits, EDNS payload, DNSSEC policy, listener addresses, and shutdown. Hickory supplies the complex iterative resolver machinery behind typed configuration and handler interfaces.

Every operator-controlled dimension is bounded. Cache sizes, recursion depth, name-server recursion depth, network lists, timeouts, and TCP message sizes have explicit limits. The `bounded_env` generic converts an environment value and verifies its range before server construction. A C implementation can do the same checks, but Rust makes the parsed type and the allowed range part of one reusable function.

Composition also improves performance engineering. The custom authoritative path stays small and directly benchmarkable. The resolver can use Tokio and a mature async DNS implementation without imposing that runtime on tinydns, rbldns, or walldns. Different concurrency models remain behind process and library boundaries rather than forcing one architecture across the suite.

25 Tests as executable protocol commentary

The strongest claims in this book have executable counterparts. `tests/rfc_conformance.rs` names normative requirements and constructs exact packets. `tests/wire_security.rs` contains a hostile corpus and checks every truncation of a structured packet. `tests/packet_properties.rs` generates arbitrary bytes and structured messages:

```
#[test]
fn arbitrary_packets_never_panic(
    bytes in prop::collection::vec(any::<u8>(), 0..4096)
) {
    let _ = Message::decode(&bytes);
}
```

The property suite does not prove that every accepted DNS packet has the desired meaning. It establishes three valuable invariants over a large input space: decoding never panics, accepted packets can be re-encoded and reparsed, and generated structured messages round-trip without semantic loss.

Golden CDB fixtures protect historical compatibility. Live UDP/TCP tests exercise connection reuse and framing. `drill` provides an independent encoder and decoder. The stable-Rust benchmark in `benches/dns_core.rs` measures the functions that rgbdns itself owns, and `docs/performance.md` records both timings and wire size.

This is where Rust most clearly changes the economics of a C rewrite. Memory safety removes many failure modes before testing. Property tests can then spend their budget on protocol structure rather than rediscovering use-after- free and buffer-overrun variants. The remaining failures are more likely to be interesting DNS mistakes.

26 Abstractions and performance ledger

The rewrite's gains are not a single "Rust is faster" claim. Some changes buy safety, some buy clarity, and some measurably improve a hot path.

Design move	Rust expression	Operational effect
Valid names at construction	private fields, `FromStr`, `Result`	invalid labels cannot circulate
Complete DNS states	`RecordType`, `RData`, `Lookup` enums	unknown types survive; negative answers stay distinct
Bounded packet access	borrowed slices and checked indexing	malformed packets fail without memory corruption
Shared immutable service state	borrowing and `Arc`	explicit thread-safe ownership
Compatibility quarantine	checked CDB decoder into owned values	old files do not become trusted memory
Independent resource limits	typed bounded configuration	one limit cannot silently stand in for another
All-node zone index	`BTreeSet<Name>`	NXDOMAIN lookup improved about 11×
Compressed writer	bounded suffix reuse	repeated-owner answer became 50.7% smaller
Binary-search truncation	monotone fit search	200-record truncation improved 17%
Thin binaries	library functions plus small adapters	easier tests and independent supervision

The encoder example is especially important. Rust did not automatically make it faster: compression made encoding 2.3 times slower than the uncompressed baseline. But it halved the measured wire size, reduced fragmentation risk, and sharply improved the complete authoritative response path. Good systems engineering reports the trade rather than reducing it to a language slogan.

The deeper improvement over C is control. The data model says what is valid, the compiler checks ownership, the boundaries return typed failure, the tests state protocol properties, and benchmarks expose the remaining costs. That combination makes rgbdns easier to change without making DNS easier to fool.

27 Where DNS ends

DNS establishes named, cacheable facts and, with DNSSEC, their authenticated origin. It does not prove that the address belongs to the application a user intended, encrypt the subsequent connection, guarantee freshness inside the TTL window, or choose a healthy endpoint. TLS identity, application discovery, load balancing, routing, and monitoring build on DNS but remain separate systems.

That boundary is the best final replacement for the phone-book metaphor. DNS is a delegated publication and discovery protocol. Its tree assigns authority; its records carry typed statements; its TTLs make caching explicit; its packet format makes efficient exchange possible; recursion joins many authorities into one answer; DNSSEC authenticates the chain; and supervision keeps the implementing processes available without becoming part of the protocol.

rgbdns expresses those ideas as small programs over shared, validated Rust types. Understanding the protocol makes the program family unsurprising. Reading the program family, in turn, shows how the abstract DNS model becomes bounded packets, immutable databases, iterative queries, atomic files, and foreground processes.

28 Appendix A: Configuration quick reference

Common daemon variables include:

Variable	Meaning
`IP`	listen address
`PORT`	listen port
`DATA`	authoritative text or CDB path where supported
`ALLOW_NETS`	comma-separated client CIDRs for recursion or transfer
`DNSCACHEIP`	recursive endpoints used by client tools and ANAME flattening
`CACHESIZE`	bounded recursive response-cache capacity
`NSCACHESIZE`	bounded nameserver-cache entries
`RECURSION_LIMIT`	ordinary recursion depth
`NS_RECURSION_LIMIT`	nameserver-resolution recursion depth
`ROOT`	djbdns-compatible resolver configuration root
`DNSSEC_POLICY`	path to the one-line-per-zone `K`/`U` signing policy

The authoritative DNSSEC utilities compose without a manager process:

```
rgbsec-keygen zone keyfile
rgbsec-sign [data [data.signed]]
rgbsec-data [data [data.cdb]]
rgbsec-ds 'Kzone:keyfile:13:validity:refresh:skew'
rgbsec-check [data.cdb [dnssec-policy]]
```

Use the command's `*-conf` generator as a starting point, then adapt the foreground `run` contract to the chosen native supervisor.

29 Appendix B: Further reading

The protocol is defined across many RFCs. A productive sequence is:

- RFC 1034, *Domain Names—Concepts and Facilities.*
- RFC 1035, *Domain Names—Implementation and Specification.*
- RFC 2181, clarifications including RRset and credibility rules.
- RFC 2308, negative caching.
- RFC 6891, Extension Mechanisms for DNS (EDNS(0)).
- RFC 7766, DNS over TCP requirements.
- RFC 4033, RFC 4034, and RFC 4035, DNSSEC.
- RFC 5155, NSEC3.
- RFC 5936, AXFR.
- RFC 1982, serial number arithmetic.

Implementation and operational references:

- The djbdns documentation: https://cr.yp.to/djbdns.html
- TAI64N format and tools: https://cr.yp.to/daemontools/tai64n.html
- s6 overview: https://skarnet.org/software/s6/overview.html
- s6-rc overview: https://skarnet.org/software/s6-rc/overview.html
- runit benefits: https://smarden.org/runit/benefits.html
- systemd project documentation: https://systemd.io/
- Hickory DNS: https://hickory-dns.org/

30 Appendix C: Submitted Internet-Draft: ANAME and Zone Transfer

The following is the complete publication text of `draft-khrabrov-dnsop-aname-axfr-00`, submitted on 30 July 2026. The live Datatracker record is https://datatracker.ietf.org/doc/draft-khrabrov-dnsop-aname-axfr/. It is an active individual Internet-Draft—a work in progress—not an approved RFC or an IETF endorsement. Pagination form-feed characters from the submitted text artifact have been rendered as blank lines; the words and page furniture are otherwise reproduced in full.

Domain Name System Operations
Internet-Draft
Intended status: Standards Track
Expires: 31 January 2027

A. Khrabrov
QueryGraph

30 July 2026

Address-specific DNS Aliases (ANAME) and Zone Transfer
draft-khrabrov-dnsop-aname-axfr-00

Abstract

This document defines the ANAME DNS resource record. ANAME provides name-to-name indirection for address queries while allowing other resource record types to exist at the same owner name. It is therefore usable at a zone apex.

This document also defines authoritative processing, TTL and failure behavior, DNSSEC considerations, and interoperable transport of ANAME records in full and incremental zone transfers. In particular, each ANAME-capable authoritative server resolves the transferred target independently. This avoids treating transient, synthesized address records as the portable source of zone data.

Discussion Venue

This note is to be removed before publishing as an RFC.

Source for this draft and an issue tracker are available at https://github.com/querygraph/rgbdns/tree/master/ietf.

Status of This Memo

Copyright Notice

Table of Contents

1. Introduction

A CNAME record redirects all queries at its owner name and, with limited DNSSEC exceptions, cannot coexist with other data. A zone apex necessarily has SOA and NS records, so a CNAME cannot provide portable apex indirection. Static A and AAAA records do not solve the operational problem when a hosting or content-delivery provider controls a dynamic set of addresses.

DNS hosting systems have independently deployed solutions called ALIAS, ANAME, apex alias, Route 53 alias, and CNAME flattening. Their common behavior is to resolve a configured target and answer the owner's A and AAAA queries using the target's addresses. Their control data, transfer behavior, failure behavior, and TTL policies differ.

Cloudflare describes CNAME flattening as following a CNAME chain and returning the final addresses rather than the CNAME [CLOUDFLARE-FLATTENING]. DNSimple describes its ALIAS as a dynamic A/AAAA lookup performed by its authoritative service [DNSIMPLE-ALIAS]. Amazon Route 53 alias records can be placed at an apex but are restricted to selected AWS resources and records [ROUTE53-ALIAS]. IBM NS1 documents an authoritative ALIAS pseudo-record that is not included in outgoing zone transfers [NS1-ALIAS]. PowerDNS can either transfer its private ALIAS representation or expand it to addresses, with explicit warnings about refresh and serial-number behavior [POWERDNS-ALIAS].

This specification replaces those non-portable control-plane differences with one RR type and defines its zone-transfer behavior. It builds on the expired DNSOP ANAME proposal [I-D.ietf-dnsop-aname]. The principal change is that the ANAME RR itself is the portable zone data: it is carried by AXFR and IXFR, and every capable authoritative server resolves it. Synthesized address RRsets are derived state, not a substitute for transferring the ANAME.

2. Requirements Language

The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, NOT RECOMMENDED, MAY, and OPTIONAL in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

3. Terminology

This document uses DNS terminology from [RFC9499].

Address RRset: An RRset of type A or AAAA.

ANAME owner: The owner name of an ANAME RR.

ANAME target: The domain name carried in ANAME RDATA.

Sibling address RRset: An A or AAAA RRset at the ANAME owner.

Target address RRset: The terminal A or AAAA RRset obtained after resolving the ANAME target and following permitted CNAME or ANAME links.

Flattening: Producing an address RRset at the ANAME owner from a target address RRset.

Native secondary: An authoritative secondary that understands ANAME, receives it in zone transfer, and performs flattening independently.

Materializing secondary: An ANAME-oblivious secondary that serves address RRsets generated by the primary. This is a transition arrangement and has additional consistency requirements.

4. The ANAME Resource Record

ANAME has RR TYPE value TBD1. Its mnemonic is ANAME.

4.1. Presentation and Wire Format

The presentation format is:

```
owner.example.  300  IN  ANAME  target.example.net.
```

The RDATA consists of exactly one domain name, the ANAME target. Its wire encoding is the DNS name wire format from [RFC1035]. Name compression MUST NOT be used within ANAME RDATA, consistent with [RFC3597]. Relative names in a master file are resolved by the master-file parser in the same manner as CNAME targets.

4.2. Cardinality and Coexistence

An ANAME RRset MUST contain exactly one record. An ANAME MUST NOT coexist at the same owner with CNAME or another RR type whose own rules prohibit coexistence. ANAME MAY coexist with SOA, NS, MX, TXT, CAA, and other ordinary types.

Authoritative zone configuration MUST NOT contain administrator-managed sibling A or AAAA RRsets at an ANAME owner. Address RRsets generated by the procedures in this document are derived state and are not independent zone content.

Wildcard ANAME records are permitted only if an implementation applies the wildcard rules of [RFC4592] before flattening. Implementations that cannot preserve those rules MUST reject wildcard ANAME configuration.

5. Target Resolution and Address Synthesis

An ANAME-capable authoritative server performs the following procedure independently for A and AAAA:

1. Resolve the ANAME target using a recursive resolver that is not dependent on the authoritative server for the ANAME owner. The implementation MUST prevent recursion back into the same unresolved ANAME dependency.

2. Follow CNAME and ANAME links to a terminal address RRset. The implementation MUST detect loops and MUST impose configurable bounds on chain depth, response size, and resolution time.

3. Validate the response according to local DNSSEC policy. The security status of the target is discussed in Section 11.

4. Replace the terminal RRset's owner with the ANAME owner and remove signatures belonging to the target zone.

5. Set each synthesized TTL to no more than the minimum of the ANAME TTL, every followed alias TTL, and the remaining terminal address TTL.

The resulting owner and RDATA form the synthesized address RRset. An implementation MUST NOT combine addresses obtained from different generations of a target RRset.

5.1. Resolution Timing and Caching

A server MAY flatten when a query arrives, refresh in advance, or use a shared cache. It SHOULD refresh before the cached target expires and SHOULD coalesce concurrent lookups for the same target and address type.

A server MUST NOT serve a synthesized positive TTL greater than the remaining lifetime of the data from which it was derived. This prevents TTL extension at the authoritative boundary. A configured ANAME TTL acts as an upper bound, not permission to extend the target TTL.

5.2. Negative Answers and Failure

Authenticated NXDOMAIN or NODATA for the target produces NODATA for the corresponding address type at the ANAME owner. A timeout, SERVFAIL, validation failure, or malformed target response is a resolution failure, not NODATA.

On resolution failure, a server SHOULD return SERVFAIL unless it has previously validated target data eligible for bounded stale serving under [RFC8767]. Stale use MUST be bounded by local policy and MUST NOT change the target's DNSSEC security status. A server MUST NOT convert a transient failure into an indefinite empty answer.

6. Query Processing

6.1. A and AAAA Queries

For an A or AAAA query at an ANAME owner, an authoritative server MUST answer from the synthesized address RRset produced by Section 5. The response is authoritative for the ANAME owner. It MUST NOT contain a CNAME synthesized from the ANAME.

An ANAME-capable authoritative server SHOULD also place the ANAME RRset in the Answer section. ANAME-oblivious resolvers ignore the unknown RR type while using the A or AAAA RRset as usual. An implementation MAY omit ANAME from an address response when response-size or compatibility policy requires it; this does not alter the address answer.

6.2. ANAME Queries

A query for type ANAME is processed as an ordinary authoritative query. If the ANAME exists, the Answer section MUST contain it. The server SHOULD add currently available synthesized A and AAAA RRsets to the Additional section. Their absence does not mean the target lacks addresses.

6.3. Other Query Types and Delegations

ANAME has no effect on query types other than A, AAAA, ANAME, and ANY. It does not redirect MX, TXT, HTTPS, SVCB, CAA, or any other data. It MUST NOT override a zone cut. The authoritative server applies normal delegation and wildcard processing before applying ANAME synthesis.

7. Zone Transfer

The lack of common transfer semantics is a material interoperability problem in deployed flattening systems. Omitting proprietary control data prevents a secondary from reproducing behavior. Expanding only to A and AAAA records captures a transient observation and does not cause a transfer when the external target changes.

7.1. Native AXFR

In an AXFR response conforming to [RFC5936], the primary MUST include each ANAME RR in zone scope exactly as it includes any other authoritative RR. The primary MUST NOT suppress ANAME merely because the requester did not advertise support for it. Unknown-RR handling in [RFC3597] permits transfer and storage by software that does not yet understand its semantics.

A native secondary MUST store the received ANAME RR and perform Section 5 using its own resolver and cache. It MUST NOT assume that an address RRset materialized at the primary remains current. Consequently, changes to external target addresses do not require the zone serial to change and do not require another transfer when all authoritative servers are native.

A primary SHOULD NOT include derived sibling A or AAAA RRsets in native AXFR. If it does include them for transition, a native secondary MUST treat them as replaceable cache seeds, bounded by their TTL, rather than administrator-managed zone data.

7.2. Native IXFR

IXFR [RFC1995] adds, deletes, or replaces the ANAME RR like any other RR. An ANAME target change is a zone-content change and MUST be accompanied by an SOA serial change. Refresh of addresses beneath an unchanged external target is derived state and MUST NOT by itself require an IXFR delta.

7.3. Capability Signaling

Operators need to know whether every authoritative server will synthesize addresses. This document defines the ANAME-CAPABLE EDNS option, using the option format defined by [RFC6891], with code TBD2 for AXFR and IXFR requests.

The option is encoded as follows:

```
+----------------------+----------------------+
| OPTION-CODE = TBD2   | OPTION-LENGTH = 0    |
+----------------------+----------------------+
     16 bits                16 bits
```

OPTION-CODE is the two-octet network-order value TBD2 and OPTION-LENGTH is the two-octet value zero. The option contains no OPTION-DATA in this version. A receiver MUST ignore an ANAME-CAPABLE option whose length is not zero.

A native secondary SHOULD include ANAME-CAPABLE in its transfer request. A primary that recognizes it SHOULD include the same empty option in the first transfer response. The option is an assertion of processing capability, not authorization, and does not alter which authoritative RRs are transferred.

Absence of ANAME-CAPABLE does not permit omission of ANAME from a standards-conforming transfer. It tells the operator that the secondary might only store the unknown RR and might not answer address queries correctly. A primary SHOULD expose this condition through logging or management telemetry.

7.4. Transition with ANAME-oblivious Secondaries

A zone using ANAME SHOULD NOT be delegated to an authoritative server that cannot synthesize it. During transition, an operator may configure the primary to materialize A and AAAA RRsets for such a secondary. This is an operational compatibility mode, not native ANAME transfer.

In compatibility mode, the primary MUST refresh the target, commit changed sibling addresses as zone changes, increment the SOA serial, and use DNS NOTIFY [RFC1996] or an equivalently prompt transfer mechanism. Merely expanding addresses during occasional AXFR while leaving the serial unchanged is NOT RECOMMENDED, because the secondary has no signal that the external target changed.

DNSSEC signing of materialized data occurs after flattening and before transfer. The operator is responsible for ensuring that all authoritative servers present a coherent signed zone.

7.5. Transfer Authentication

ANAME does not change AXFR authorization requirements. Zone transfers commonly disclose the complete namespace and SHOULD be restricted and authenticated using TSIG [RFC8945], SIG(0), mutually authenticated transport, or an equivalent mechanism. ANAME-CAPABLE is not an authentication mechanism.

8. Examples

8.1. Native Primary and Secondary

The operator configures the following authoritative data:

```
example.  3600  IN  SOA    ns1.example. hostmaster.example. (
                             2026073001 3600 900 1209600 300 )
example.  3600  IN  NS     ns1.example.
example.  3600  IN  NS     ns2.example.
example.   300  IN  ANAME  service.example.net.
example.  3600  IN  MX 10  mail.example.
```

The AXFR contains the SOA bookends, NS, MX, and ANAME records. It does not need to contain synthesized A or AAAA records. The native secondary advertises ANAME-CAPABLE and stores the ANAME. Each authoritative server independently resolves service.example.net..

If the target's A RRset has TTL 120 and contains 192.0.2.10, an A query can receive:

```
example.  120  IN  A      192.0.2.10
example.  300  IN  ANAME  service.example.net.
```

The synthesized A TTL is 120, the smaller of the remaining target TTL and the configured ANAME ceiling. The ANAME retains its own TTL. The MX record and all other owner data remain unaffected.

If the target later changes to 192.0.2.20 without any change to the example. zone, both native servers refresh independently. The SOA serial need not change and no transfer is required.

8.2. Changing the Target

If the administrator changes the target itself to new-service.example.net., the ANAME is authoritative zone content. The primary increments the SOA serial and transfers this deletion and addition through IXFR, or sends the complete new ANAME through AXFR. This differs from an address change beneath an unchanged target.

8.3. Legacy Materialization

Suppose ns2.example. does not synthesize ANAME. If the operator temporarily uses compatibility mode, the primary commits the current 192.0.2.10 A record as derived zone data for that transfer. When the target changes to 192.0.2.20, the primary must refresh, replace the committed A RRset, increment the serial, notify the secondary, and transfer the change. Expanding only when an unrelated AXFR happens can leave ns2.example. stale indefinitely.

9. DNSSEC

This section uses the DNSSEC protocol and terminology defined by [RFC4033], [RFC4034], and [RFC4035].

The ANAME RRset is authoritative zone data and MUST be signed when the zone is signed. Synthesized address RRsets are authoritative statements by the ANAME owner's zone; signatures from the target zone cannot be copied because the owner name differs.

A server using online synthesis needs online signing if it returns signed synthesized address RRsets. A server using offline signing MUST materialize, sign, and publish address changes before their previous signatures or operational validity expire. A signed ANAME with no validly signed synthesized address response can cause validating clients to treat an answer as bogus.

Signing synthesized addresses asserts that the ANAME owner's authority chose those addresses; it does not preserve the target zone's chain of trust. Implementations SHOULD validate the target before synthesizing signed data and MUST provide a policy for insecure, bogus, and indeterminate targets.

10. Operational Considerations

10.1. Multi-provider Consistency

Native authoritative servers resolve from different network locations and caches. A target using geographic, latency, load, or EDNS Client Subnet policy can therefore yield different, individually valid address RRsets. Exact address equality among authorities is not required. Operators SHOULD test that every result is valid for clients and that TTL and failure policies are compatible.

Authoritative flatteners SHOULD NOT forward a client's EDNS Client Subnet value by default. If they do, they MUST follow [RFC7871] and account for the resulting cache variance and privacy exposure.

10.2. Monitoring

Implementations SHOULD expose target resolution latency, cache age, refresh failure, DNSSEC status, stale-answer use, chain depth, synthesized TTL, and transfer capability. Operators SHOULD query A and AAAA independently at every authoritative server.

11. Security Considerations

ANAME makes authoritative serving depend on recursive resolution. Implementations MUST isolate that resolver from authoritative query processing sufficiently to prevent deadlock and unbounded resource consumption. They MUST bound target chain length, response size, address count, lookup concurrency, retry rate, and total lookup time.

A target controlled by another party can redirect an ANAME owner to arbitrary addresses. This is the intended delegation of address control, but it has security consequences similar to granting that party permission to edit A and AAAA records. Zone administrators SHOULD verify target ownership and lifecycle, and remove ANAME records before relinquishing a target name.

Resolvers used for flattening MUST defend against cache poisoning and bailiwick violations. DNSSEC validation SHOULD be enabled. An implementation MUST NOT silently promote a bogus target into signed synthesized data.

Self-reference and multi-name loops can cross zone boundaries. Implementations MUST detect loops within one resolution operation. They SHOULD coalesce concurrent lookups for the same target and retain short-lived failure state to suppress repeated cyclic work. On loop detection they MUST terminate the active chain and return SERVFAIL; they MUST NOT start a fresh recursive lookup that discards the known chain context. These requirements limit geometric query amplification when a loop crosses multiple authoritative providers.

ANAME targets may resolve to private, loopback, link-local, or other special-use addresses. DNS itself permits such answers, so filtering is a deployment policy. Implementations SHOULD offer policy controls and clear diagnostics, especially where resolver traffic crosses trust boundaries.

The transfer capability option is unauthenticated and can be removed or forged by an on-path attacker. It MUST NOT be used to authorize transfer or as the sole basis for a security decision. Authenticated transfer protects both the ANAME target and other zone contents.

12. IANA Considerations

IANA is requested to assign a value from the "Resource Record (RR) TYPEs" subregistry of the "Domain Name System (DNS) Parameters" registry as follows:

Type	Value	Meaning	Reference
ANAME	TBD1	Address-specific DNS alias	This document

Table 1

IANA is requested to assign a value from the "DNS EDNS0 Option Codes (OPT)" registry as follows:

Name	Value	Status	Reference
ANAME-CAPABLE	TBD2	Standard	This document

Table 2

13. Implementation and Deployment Experience

This section is to be removed before publication as an RFC.

rgbdns 0.2.3 implements on-demand authoritative flattening, bounded caching, concurrent-miss coalescing, short-lived failure suppression, TTL capping, CNAME-chain and loop handling, per-zone native transfer, and independent target resolution by its secondary. Before IANA assignment, it uses an explicitly negotiated experimental protocol: EDNS option 65001 containing the four octets RGA1, and private-use TYPE65401 whose RDATA is RGA1 followed by the uncompressed target DNS name. The private RR TTL carries the configured ANAME TTL ceiling.

A standard AXFR client does not receive rgbdns's experimental record. This behavior prevents private metadata from being mistaken for an assigned RR, but it is not the final behavior specified in Section 7.1. An implementation of this document will use TBD1 directly and transfer it to all AXFR clients. During migration, rgbdns can accept both its versioned private encoding and the assigned ANAME RR, but MUST NOT emit TYPE65401 as if it were the assigned standard.

PowerDNS also uses private-use TYPE65401 for ALIAS, but its RDATA is an unprefixed DNS name [POWERDNS-ALIAS]. The two encodings are not interoperable despite sharing a private-use value. This collision demonstrates why a private-use RR value cannot establish a cross-vendor standard and why receivers must not infer semantics from TYPE65401 without out-of-band agreement.

14. Changes from draft-ietf-dnsop-aname-04

This section is to be removed before publishing as an RFC.

* Makes the ANAME RR, rather than synthesized sibling addresses, the portable source data for native AXFR and IXFR.

* Requires each native authoritative server to resolve the target, eliminating transfer-driven refresh for external address changes.

* Adds ANAME-CAPABLE signaling for operational detection without making transfer contents depend on negotiation.

* Separates native transfer from a serial-changing materialization mode for legacy secondaries.

* Specifies failure, stale-answer, bounds, monitoring, multi-provider, and migration behavior informed by deployed services.

* Adds end-to-end examples distinguishing a changed ANAME target from changed addresses beneath an unchanged target.

* Documents the rgbdns implementation and the TYPE65401 collision with PowerDNS.

15. Acknowledgments

This document builds on draft-ietf-dnsop-aname by Tony Finch, Evan Hunt, Peter van Dijk, Anthony Eden, and Willem Mekking. Their work and the DNSOP discussions established the ANAME model, terminology, and much of the analysis of TTLs, DNSSEC, and alias chains.

Deployment documentation from Cloudflare, DNSimple, Amazon Web Services, IBM NS1, and PowerDNS supplied concrete evidence about flattening behavior and zone-transfer interoperability.

16. Normative References

[RFC1035] Mockapetris, P., "Domain names - implementation and specification", STD 13, RFC 1035, DOI 10.17487/RFC1035, November 1987, <https://www.rfc-editor.org/info/rfc1035>.

[RFC1995] Ohta, M., "Incremental Zone Transfer in DNS", RFC 1995, DOI 10.17487/RFC1995, August 1996, <https://www.rfc-editor.org/info/rfc1995>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC3597] Gustafsson, A., "Handling of Unknown DNS Resource Record (RR) Types", RFC 3597, DOI 10.17487/RFC3597, September 2003, <https://www.rfc-editor.org/info/rfc3597>.

[RFC4033] Arends, R., Austein, R., Larson, M., Massey, D., and S. Rose, "DNS Security Introduction and Requirements", RFC 4033, DOI 10.17487/RFC4033, March 2005, <https://www.rfc-editor.org/info/rfc4033>.

[RFC4034] Arends, R., Austein, R., Larson, M., Massey, D., and S. Rose, "Resource Records for the DNS Security Extensions",

RFC 4034, DOI 10.17487/RFC4034, March 2005,
<https://www.rfc-editor.org/info/rfc4034>.

[RFC4035] Arends, R., Austein, R., Larson, M., Massey, D., and S. Rose, "Protocol Modifications for the DNS Security Extensions", RFC 4035, DOI 10.17487/RFC4035, March 2005, <https://www.rfc-editor.org/info/rfc4035>.

[RFC4592] Lewis, E., "The Role of Wildcards in the Domain Name System", RFC 4592, DOI 10.17487/RFC4592, July 2006, <https://www.rfc-editor.org/info/rfc4592>.

[RFC5936] Lewis, E. and A. Hoenes, "DNS Zone Transfer Protocol (AXFR)", RFC 5936, DOI 10.17487/RFC5936, June 2010, <https://www.rfc-editor.org/info/rfc5936>.

[RFC6891] Damas, J., Graff, M., and P. Vixie, "Extension Mechanisms for DNS (EDNS(0))", STD 75, RFC 6891, DOI 10.17487/RFC6891, April 2013, <https://www.rfc-editor.org/info/rfc6891>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/info/rfc8174>.

[RFC8945] Dupont, F., Morris, S., Vixie, P., Eastlake, D., and O. Gudmundsson, "Secret Key Transaction Authentication for DNS (TSIG)", RFC 8945, DOI 10.17487/RFC8945, November 2020, <https://www.rfc-editor.org/info/rfc8945>.

[RFC9499] Hoffman, P. and K. Fujiwara, "DNS Terminology", BCP 219, RFC 9499, DOI 10.17487/RFC9499, March 2024, <https://www.rfc-editor.org/info/rfc9499>.

17. Informative References

[CLOUDFLARE-FLATTENING]
Cloudflare, "CNAME flattening", Cloudflare DNS

documentation, 24 June 2026,
<https://developers.cloudflare.com/dns/cname-flattening/>.

[DNSIMPLE-ALIAS]
DNSimple, "What Is an ALIAS Record?", DNSimple Help, 2026,
<https://support.dnsimple.com/articles/alias-record/>.

[I-D.ietf-dnsop-aname]
Finch, T., Hunt, E., van Dijk, P., Eden, A., and W. Mekking, "Address-specific DNS aliases (ANAME)", Work in Progress, Internet-Draft, draft-ietf-dnsop-aname-04, 8 July 2019, <https://datatracker.ietf.org/doc/html/draft-ietf-dnsop-aname-04>.

[NS1-ALIAS]
IBM, "Comparing CNAME, ALIAS, and linked records", IBM NS1 Connect documentation, 2026, <https://www.ibm.com/docs/en/ns1-connect?topic=answers-comparing-cname-alias-linked-records>.

[POWERDNS-ALIAS]
PowerDNS.COM BV, "Using ALIAS records", PowerDNS Authoritative Server documentation, 2026,
<https://doc.powerdns.com/authoritative/guides/alias.html>.

[RFC1996] Vixie, P., "A Mechanism for Prompt Notification of Zone Changes (DNS NOTIFY)", RFC 1996, DOI 10.17487/RFC1996, August 1996, <https://www.rfc-editor.org/info/rfc1996>.

[RFC7871] Contavalli, C., van der Gaast, W., Lawrence, D., and W. Kumari, "Client Subnet in DNS Queries", RFC 7871, DOI 10.17487/RFC7871, May 2016,
<https://www.rfc-editor.org/info/rfc7871>.

[RFC8767] Lawrence, D., Kumari, W., and P. Sood, "Serving Stale Data to Improve DNS Resiliency", RFC 8767, DOI 10.17487/RFC8767, March 2020,
<https://www.rfc-editor.org/info/rfc8767>.

[ROUTE53-ALIAS]

Amazon Web Services, "Choosing between alias and non-alias records", Amazon Route 53 Developer Guide, 2026, <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-sets-choosing-alias-non-alias.html>.

Author's Address

Alexy Khrabrov
QueryGraph
Email: deliverable@gmail.com
URI: https://github.com/querygraph/rgbdns